



SPONGE: Adaptive Boundary-Anchored Indexing for Online Value-Flow Queries

SIXIANG PENG, Hong Kong University of Science and Technology, China

CHENYANG SUN, Hong Kong University of Science and Technology, China

WEI CHEN, Hong Kong University of Science and Technology, China

BOWEN ZHANG, Hong Kong University of Science and Technology, China

CHARLES ZHANG[✉], Hong Kong University of Science and Technology, China

The application of high-precision value-flow analysis is experiencing a paradigm shift from planned executions to online ad hoc queries driven by human auditors and AI agents. However, existing techniques struggle in this interactive setting: exhaustive offline tabulation is fundamentally intractable, while memoryless online search suffers from redundant exploration and SMT invocations. To bridge this gap, we propose SPONGE, a novel two-phase framework that accelerates ad hoc queries through boundary-anchored indexing. Offline, SPONGE employs an adaptive-depth strategy to selectively precompute feasible value-flow segments at critical procedure boundaries, optimizing SMT allocation based on traversal probability and search space complexity. Online, it utilizes an index-guided push-down search with lazy expansion to dynamically stitch these pre-verified segments, effectively bypassing redundant state exploration and pruning unsatisfiable paths. We evaluated SPONGE on 9 C/C++ projects (up to 3.8 million LoC). Results demonstrate that SPONGE drops the 95th-percentile online query time from nearly 270 s to under 50 s compared to a baseline search. Furthermore, the adaptive strategy reduces offline indexing time by 75% over a uniform approach, amortizing the offline cost in fewer than 300 queries for workloads dominated by complex queries.

CCS Concepts: • **Theory of computation** → **Program analysis**; • **Software and its engineering** → **Software safety**; **Software testing and debugging**.

Additional Key Words and Phrases: Static Analysis, Value-Flow Analysis, Online Queries, Indexing, SMT Solving

ACM Reference Format:

Sixiang Peng, Chenyang Sun, Wei Chen, Bowen Zhang, and Charles Zhang. 2026. SPONGE: Adaptive Boundary-Anchored Indexing for Online Value-Flow Queries. *Proc. ACM Program. Lang.* 10, OOPSLA2, Article 340 (October 2026), 28 pages. <https://doi.org/10.1145/3839472>

1 Introduction

The concept of value flow, which characterizes how values propagate across assignments, pointer dereferences, and procedure boundaries, serves as a fundamental substrate for static analysis [10, 26, 35, 43, 49, 52]. Making this propagation explicit supports critical tasks, including vulnerability discovery [10, 43, 51], program understanding [16, 60], and program repair [17]. Modern approaches achieve scalability by precomputing def-use connectivity to construct a sparse *value-flow graph* [43, 50].

Authors' Contact Information: Sixiang Peng, Hong Kong University of Science and Technology, Hong Kong, China, speng-gaf@cse.ust.hk; Chenyang Sun, Hong Kong University of Science and Technology, Hong Kong, China, csunap@cse.ust.hk; Wei Chen, Hong Kong University of Science and Technology, Hong Kong, China, wchenbt@cse.ust.hk; Bowen Zhang, Hong Kong University of Science and Technology, Hong Kong, China, bzhangbr@cse.ust.hk; Charles Zhang, Hong Kong University of Science and Technology, Hong Kong, China, charlesz@cse.ust.hk.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/10-ART340

<https://doi.org/10.1145/3839472>

While the value-flow graph provides a scalable foundation, making the results actionable requires *high-precision witnesses*. A valid witness must correspond to a *feasible path* [43, 51, 57] that satisfies two rigorous constraints: it must be *realizable* in its control flow (valid caller-callee matching via extended Dyck-CFL reachability [41]), and *satisfiable* in its branch conditions (resolvable by an SMT solver).

The Paradigm Shift: Planned versus Ad Hoc Queries. Value-flow analysis operates in two distinct paradigms. The traditional paradigm relies on *planned queries*, such as nightly bug detection [38], where a predefined set of source–sink (s, t) pairs can be globally orchestrated to optimize aggregate throughput. In contrast, we target an *online serving scenario*: a centralized analysis service preprocesses a stable program revision during an off-peak period and subsequently serves multiple downstream clients, such as security auditors [34, 53] and autonomous agents [27, 54]. Each client issues ad hoc (s, t) queries that are not known during preprocessing and arrive independently. Consequently, the primary objective shifts from the total processing time of a fixed query batch to per-query response time, particularly tail latency across requests. This separation motivates a query-independent offline index whose construction cost can be amortized across clients while retaining flexibility for unforeseen queries.

Limitations of Existing Approaches. Despite extensive research on value-flow analysis, adapting existing techniques to support high-precision, online ad hoc (s, t) queries remains inherently difficult. We observe two primary adaptation strategies based on current methodologies, but both fall short in an interactive querying scenario.

The first strategy adapts standard tabulation-based algorithms, which achieve context sensitivity by computing function summaries bottom-up along the call graph [2, 5, 30, 43, 51]. While effective for planned queries where targets are predefined, generalizing this mechanism to build a universal offline index is fundamentally intractable due to the sheer volume of feasible value flows. Consequently, ad hoc (s, t) queries force the analysis to execute from scratch online. This re-execution exposes a critical inefficiency: the bottom-up nature of tabulation requires eagerly summarizing callees before their callers. Even if restricted to the queried values, the algorithm inevitably explores and summarizes vast, irrelevant program regions that never participate in a feasible path between s and t , inducing prohibitive latency for interactive use.

To bypass the irrelevant computations of bottom-up tabulation, a natural alternative is a *push-down, candidate-then-verify* Depth-First Search (DFS) [37, 46–48]. Starting directly from the queried endpoint s , this approach tracks calling contexts to extract realizable paths, which an SMT solver subsequently verifies. While restricting the search space to paths originating from s avoids exhaustive summarization, it suffers from severe redundant computation. Because ad hoc queries are typically isolated and independent, standard inter-query caching yields negligible benefits, and determining optimal intermediate states to cache is highly non-trivial. Consequently, standard DFS must blindly re-explore callees and repeatedly invoke the SMT solver whenever the same program regions are reached under different contexts. This complete absence of computational reuse severely amplifies the inherent *path explosion* problem [3, 8].

In summary, existing techniques offer complementary strengths but leave a gap for online ad hoc queries. Summary-based analyses reuse computation within predefined analysis domains, but their summaries are not designed as query-independent feasibility evidence for arbitrary endpoints. Push-down search naturally supports arbitrary endpoints, but recomputes path feasibility across independent queries. This raises two design questions: what semantic information can be reused independently of future queries, and where should limited offline effort be spent?

Our Approach: Adaptive Boundary-Anchored Indexing. To bridge the gap between exhaustive summarization and memoryless search, we propose a two-phase framework driven by two key insights regarding value-flow paths.

First, about **What to Index**: We observe that the core overhead of value flow queries lies in solving the SMT of all candidate paths. Notably, a value flow path can be divided into several segments, and the feasibility of these segments is a necessary condition for the feasibility of the entire path. Therefore, we consider computing and storing several feasible segments offline. Then, for a specific online source-sink query, we can obtain fewer candidate paths by concatenating these feasible segments, thus effectively avoiding redundant computation.

Second, about **How to select the Index**: We note that due to limited storage resources, it is impractical to offline index all feasible segments. Further, we observe that there are some “busy” nodes in the Value Flow Graph. From the perspective of incoming edges, they have a high probability of being accessed; from the perspective of outgoing edges, they lead to dense successor paths. Obviously, segments near these busy nodes are more critical, and indexing them will bring more benefits to online queries.

Based on these insights, we design an *adaptive, boundary-anchored indexing mechanism* that maximizes the utility of offline computations through a synergistic two-phase framework:

- **The Boundary-Anchored Index**: We generalize function summaries to capture the witnesses and SMT satisfiability of feasible paths between arbitrary procedure boundary nodes (BNs), such as arguments or callsite outputs. Anchoring at BNs cleanly separates dense intra-procedural propagation from sparse inter-procedural transitions. This index allows us to structurally stitch pre-verified segments into high-quality candidates while proactively pruning unsatisfiable branches.
- **Offline Phase (Adaptive-Depth Construction)**: To ensure scalability, we introduce an *adaptive-depth indexing strategy*. Instead of exploring paths to a uniform depth, we dynamically allocate SMT resources based on a cost-benefit strategy for each BN. By weighing a BN’s search complexity against its traversal probability, we concentrate offline computation strictly on the most critical boundaries.
- **Online Phase (Index-Guided Search with Lazy Expansion)**: During an (s, t) query, we employ an index-guided push-down DFS. When the search encounters BNs, it leverages the index to “jump” across complex inter-procedural logic. If a segment remains unindexed due to the exhausted precomputed depth limit, it triggers *lazy expansion* to on-the-fly compose shorter cached witnesses into longer extensions, drastically reducing redundant state exploration and SMT calls.

Implementation and Contributions. We implemented our approach in SPONGE for C/C++ and evaluated it on 9 SPEC2017 projects (up to 3.8 million LoC). For complex ad hoc queries, SPONGE reduces expensive online SMT invocations by up to 85% and drops the 95th-percentile (P95) response time from approximately 270 s to under 50 s compared to a baseline push-down DFS, nearly eliminating timeouts. Additionally, our adaptive-depth strategy cuts offline indexing time by 75% compared to a uniform approach, with the offline cost amortizing in fewer than 300 queries for workloads dominated by complex queries.

In summary, this paper makes the following contributions:

- A novel two-phase paradigm for *online ad hoc value-flow queries*, which precomputes a composable *boundary-anchored index* to avoid exhaustive online tabulation.
- An *adaptive-depth precomputation* strategy that optimizes offline SMT resources allocation at bottleneck boundaries based on their visit probability and search space.

- An *index-guided push-down DFS* with lazy expansion, which structurally stitches indexed segments on the fly to efficiently prune unsatisfiable paths.
- An implementation (SPONGE) and evaluation demonstrating a drop in P95 query time from nearly 270 s to under 50 s over a baseline push-down DFS, and 75% lower offline time compared to a uniform-depth approach.

2 Background

This section establishes the formal foundation of our approach. We define the guarded value-flow graph (§ 2.1), formalize the criteria for feasible paths which serve as analysis witnesses (§ 2.2), and contrast traditional exhaustive analysis with our online query formulation (§ 2.3).

2.1 Sparse Value-Flow Analysis

Sparse value-flow analysis abstracts program semantics into a directed graph. We first outline the core language syntax and then define the graph construction.

2.1.1 Core Language Syntax. Figure 1 presents the syntax of our core call-by-value language [32]. To facilitate precise value-flow tracking, we assume the input program is in Static Single Assignment (SSA) form [13], ensuring each variable possesses a unique definition site. Furthermore, we unroll loops up to a predefined bound to statically handle cyclic control flows and guarantee analysis termination, a common abstraction widely adopted by modern static analyzers [10, 43] and bounded model checkers [22].

Program	P	$::= F+$
Function	F	$::= f(v_1, v_2, \dots)\{S;\}$
Statement	S	$::= v_1 = L \mid v_1 = v_2 \mid v_1 = v_2 \oplus v_3$ $\mid v_1.a = v_2 \mid v_1 = v_2.a \mid v_1 = *v_2 \mid *v_1 = v_2$ $\mid S_1; S_2 \mid \text{if } (v_1 \odot v_2)\{S_1;\} \text{ else } \{S_2;\}$ $\mid v_r = f(v_1, v_2, \dots) \mid \text{return } v$
Arithmetic	$\oplus$	$::= + \mid - \mid \times \mid \div$
Comparison	$\odot$	$::= \equiv \mid \neq \mid > \mid <$
Variable	V	$::= v_1 \mid v_2 \mid v_r \mid \dots$
Field	A	$::= a_1 \mid a_2 \mid \dots$
Literal	L	$::= \text{NULL} \mid l_1 \mid l_2 \mid \dots$

Fig. 1. Core language syntax.

2.1.2 Guarded Value-Flow Graph. Sparse value-flow analysis abstracts program semantics into a directed graph where nodes are versioned SSA variables. This value propagation occurs via two primary mechanisms: (1) *Direct flows*, which arise from top-level assignments. For example, a direct assignment statement $v_1 = v_2$ yields an edge $v_2 \rightarrow v_1$; and (2) *Indirect flows*, which occur through memory aliases. For instance, a store statement $*v_1 = v_2$ followed by a load statement $v_3 = *v_4$ yields an edge $v_2 \rightarrow v_3$, provided that the pointer variables v_1 and v_4 may point to the same memory location. We rely on an off-the-shelf pointer analysis [26, 43, 51] to resolve alias relations upfront, materializing indirect memory accesses as explicit value-flow edges.

To ensure the precision of the sparse analysis, the value-flow graph must be enriched with flow conditions and calling contexts. We leverage an existing quasi-path-sensitive analysis [43] to generate constraint formulas for the edges. Furthermore, based on the program's call graph, we

explicitly annotate interprocedural edges that connect actual arguments to formal parameters, and return values to call-site assignments.

Definition 2.1 (Guarded Value-Flow Graph). Let V be the set of SSA variables. A guarded value-flow graph (VFG) is a directed graph $G = (V, E)$. Each edge $e \in E$ is denoted as:

$$e = (u \xrightarrow{\gamma, \varphi} v)$$

where $u, v \in V$. The *context tag* γ ranges over the alphabet $\Sigma_C \triangleq \{\epsilon\} \cup \{\llbracket i, \rrbracket_i \mid i \in \{1, \dots, C\}\}$, with C being the number of call sites:

- ϵ denotes an intraprocedural step (e.g., $v_2 \xrightarrow{\epsilon, \text{true}} v_1$ for $v_1 = v_2$).
- $\llbracket i, \rrbracket_i$ tags a call-to-callee transition, propagating an actual argument v_a to a formal parameter v_p at call site i ($v_a \xrightarrow{\llbracket i, \rrbracket_i, \text{true}} v_p$).
- $\rrbracket_i$ tags a return-to-caller transition, propagating the return value v_{ret} to the receiver v_r at call site i ($v_{ret} \xrightarrow{\rrbracket_i, \text{true}} v_r$).

The term φ is a quantifier-free constraint formula guarding the flow (e.g., $\varphi \equiv (v_1 > v_2)$ for a true branch, or true for unconditional flows).

2.2 Feasible Path and Witness

A value flow is represented as a directed path π of length m from node u to node v , denoted as $\pi = u_0, e_1, u_1, \dots, e_m, u_m$ (or simply $\pi : u \rightsquigarrow v$). Along this path, we track its interprocedural calling context sequence $\text{tag}(\pi)$ and its accumulated logical path constraint $\text{pc}(\pi)$:

$$\text{tag}(\pi) \triangleq \gamma_1 \gamma_2 \dots \gamma_m, \quad \text{pc}(\pi) \triangleq \bigwedge_{j=1}^m \varphi_j.$$

To represent a valid runtime execution, a path must satisfy both contextual well-formedness and logical satisfiability. We first define contextual realizability.

Definition 2.2 (Realizable Path). A path $\pi : u \rightsquigarrow v$ is a *realizable path*, denoted as $\pi : u \xrightarrow{\text{cs}} v$, if its context sequence belongs to the extended Dyck-CFL grammar [21, 36], i.e., $\text{tag}(\pi) \in L_{\text{ctx}}$.

The context-free language L_{ctx} is generated by the following grammar:

$$\begin{aligned} S &\rightarrow P N \\ P &\rightarrow M P \mid \llbracket i, \rrbracket_i P \mid \epsilon \\ N &\rightarrow M N \mid \rrbracket_i N \mid \epsilon \\ M &\rightarrow \llbracket i, \rrbracket_i M \mid M M \mid \epsilon \end{aligned}$$

Here, M produces perfectly matched calls and returns, P allows unmatched returns (valid suffixes), and N allows unmatched calls (valid prefixes). A fundamental property of L_{ctx} is its closure under substring extraction.

PROPERTY 1 (SUBPATH CLOSURE). For any context sequence $\omega \in L_{\text{ctx}}$, if ω can be factored as $\omega = \alpha \cdot \omega' \cdot \beta$ for some sequences α and β , then $\omega' \in L_{\text{ctx}}$.

This standard property of Dyck languages guarantees that any subpath of a realizable path remains contextually realizable, providing the theoretical basis for offline path concatenation and index composition. However, contextual realizability alone does not guarantee execution at runtime, as conflicting branch conditions may render the path logically infeasible. By unifying contextual realizability and logical satisfiability, we define a feasible path.

Definition 2.3 (Feasible Path). A realizable path $\pi : u \xrightarrow{cs} v$ is a *feasible path*, denoted as $\pi : u \xrightarrow{ps:sat} v$, if its accumulated path constraint $pc(\pi)$ is logically satisfiable.

In practice, the satisfiability of $pc(\pi)$ is verified using an SMT solver. Paths that are contextually realizable but logically dead (i.e., $pc(\pi)$ is unsatisfiable) are denoted as *infeasible paths*, represented as $\pi : u \xrightarrow{ps:unsat} v$. Because a feasible path π is explicitly defined as the ordered sequence of nodes and edges $\{u_0, e_1, \dots, u_m\}$, it inherently serves as the concrete *witness* for practical diagnosis, providing the exact, step-by-step chain demonstrating how a value propagates from the source to the sink.

2.3 Problem Statement

Traditionally, value-flow analysis is formulated as a static, exhaustive discovery process. Given predefined sets of source nodes S and sink nodes T , traditional approaches seek to compute the universal set of all feasible paths across the entire program:

$$\Pi = \{\pi \mid \exists s \in S, \exists t \in T \text{ s.t. } \pi : s \xrightarrow{ps:sat} t\}$$

In contrast, emerging applications such as interactive security auditing and autonomous AI-driven program investigation introduce a paradigm shift toward on-demand, ad hoc querying. In such scenarios, the global sets S and T are not known *a priori*. Instead, ad hoc query pairs (s, t) arrive continuously, demanding rapid, on-the-fly responses rather than computing the entire set Π .

Online Ad Hoc Value-Flow Queries: Given a value-flow graph $G = (V, E)$ and a stream of ad hoc query pairs $(s, t) \in V \times V$ that arrive online, answer each query on demand by either (i) returning a feasible path π from s to t on G (i.e., $\pi : s \xrightarrow{ps:sat} t$), or (ii) returning $\perp$ if no such path exists.

The fundamental challenge in resolving such ad hoc queries is the prohibitive overhead of repeated SMT solver invocations during online path exploration. To mitigate this bottleneck, the core algorithmic problem reduces to maximizing the utility of an offline precomputation phase. Specifically, by indexing highly reusable, feasible path segments between procedure boundaries, the goal is to proactively prune the online search space and accelerate the discovery of π .

3 SPONGE in a Nutshell

In this section, we illustrate the core intuition of SPONGE through a motivating example, demonstrating how an adaptive boundary-anchored index effectively mitigates path explosion and eliminates redundant SMT invocations to accelerate online value-flow queries.

Motivation Example. As shown in Figure 2, the code snippet is presented in Figure 2a, and its corresponding Value-Flow Graph is illustrated in Figure 2b. Consider an online querying scenario where a stream of ad hoc query pairs arrives continuously. For example, one query might seek a feasible path from the external input x_2 to r_6 , denoted as (x_2, r_6) . Another entirely separate query in the stream might check if the null pointer y_3 can reach r_6 , denoted as (y_3, r_6) .

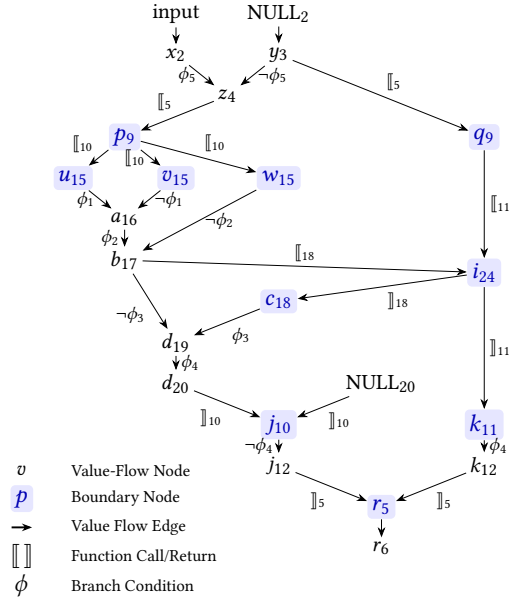
The Bottleneck of Naive DFS. Here, *Naive DFS* denotes an unindexed, on-demand push-down search over the guarded VFG. It enforces the extended Dyck-CFL calling-context discipline and invokes SMT only on complete source-sink candidates. Its limitation is therefore not context insensitivity, but the absence of reusable feasibility results across independent queries. Consider first the query (y_3, r_6) . The flow from y_3 splits into two branches: one reaching p_9 via z_4 , and the other proceeding directly to q_9 . Through p_9 , the flow enters g and takes one of three routes

```

1 void m() {
2   int* x = input();
3   int* y = NULL;
4   int* z =  $\phi_5$  ? x : y;
5   int* r = f(z, y);
6   *r = 1;
7 }
8
9 int* f(int* p, int* q) {
10  int* j = g(p, p, p);
11  int* k = h(q);
12  return  $\phi_4$  ? k : j;
13 }
14
15 int* g(int* u, int* v, int* w) {
16  int* a =  $\phi_1$  ? u : v;
17  int* b =  $\phi_2$  ? a : w;
18  int* c = h(b);
19  int* d =  $\phi_3$  ? c : b;
20  return  $\phi_4$  ? d : NULL;
21 }
22
23 int* h(int* i) {
24   return i;
25 }

```

(a) Code Example



(b) Value-Flow Graph

From	To	Depth	Feasible Paths
p_9	u_{15}	1	$p_9 \rightarrow u_{15}$
p_9	v_{15}	1	$p_9 \rightarrow v_{15}$
p_9	w_{15}	1	$p_9 \rightarrow w_{15}$
p_9	i_{24}	2	$p_9 \rightarrow \{u_{15} \rightarrow a_{16}, v_{15} \rightarrow a_{16}, w_{15}\} \rightarrow b_{17} \rightarrow i_{24}$
p_9	j_{10}	2	$p_9 \rightarrow \{u_{15} \rightarrow a_{16}, v_{15} \rightarrow a_{16}, w_{15}\} \rightarrow b_{17} \rightarrow d_{19} \rightarrow d_{20} \rightarrow j_{10}$
p_9	c_{18}	3	$p_9 \rightarrow \{u_{15} \rightarrow a_{16}, v_{15} \rightarrow a_{16}, w_{15}\} \rightarrow b_{17} \rightarrow i_{24} \rightarrow c_{18}$
p_9	j_{10}	4	$p_9 \rightarrow \{u_{15} \rightarrow a_{16}, v_{15} \rightarrow a_{16}, w_{15}\} \rightarrow b_{17} \rightarrow i_{24} \rightarrow c_{18} \rightarrow d_{19} \rightarrow d_{20} \rightarrow j_{10}$
p_9	r_5	5	$\emptyset$
q_9	i_{24}	1	$q_9 \rightarrow i_{24}$
i_{24}	k_{11}	1	$i_{24} \rightarrow k_{11}$
k_{11}	r_5	1	$k_{11} \rightarrow k_{12} \rightarrow r_5$

(c) Partial Boundary-Anchored Index

Fig. 2. A motivating example of our online value-flow query and boundary-anchored indexing. Solid arrows (“ $\rightarrow$ ”) represent value-flow edges, and blue boxes denote boundary nodes serving as offline index anchors. Edge labels “[i ,” and “ ϕ_i ” indicate function calls/returns at line i and branch conditions, respectively. The input and NULL denote external inputs and null pointers. The bottom table (c) illustrates a partial boundary-anchored index, detailing the feasible paths between selected boundary nodes at specific depths.

($u_{15} \rightarrow a_{16}$, $v_{15} \rightarrow a_{16}$, or w_{15}) before splitting again at b_{17} : directly to d_{19} or through h via $i_{24} \rightarrow c_{18}$. This yields 6 structural candidates through p_9 and one through q_9 . Thus, Naive DFS makes 7 SMT calls; only $y_3 \rightarrow q_9 \rightarrow i_{24} \rightarrow k_{11} \rightarrow k_{12} \rightarrow r_5 \rightarrow r_6$ is satisfiable. Similarly, for the query (x_2, r_6) , the flow passes through $x_2 \rightarrow z_4 \rightarrow p_9$ and cannot reach q_9 . Naive DFS enumerates 6 candidate paths, all of which are infeasible. Across both queries, 12 of 13 SMT invocations reject infeasible

candidates, several of which contain the same contradictory segment. For instance, any candidate path containing $d_{19} \rightarrow d_{20} \rightarrow j_{10} \rightarrow j_{12}$ is unsatisfiable due to the contradiction in ϕ_4 .

Boundary-Anchored Index Structure. To eliminate redundant SMT checks during the online phase, our framework introduces a boundary-anchored index. As illustrated in Figure 2c, the index records the *Feasible Paths* between BNs. These nodes, highlighted with blue backgrounds in Figure 2b (e.g., $p_9, q_9, u_{15}, i_{24}, r_5$), represent the entry points of interprocedural value flows (arguments or callsite outputs). The index categorizes these cached paths by *Depth*, which measures the number of inter-procedural boundary crossings. For example, as shown in Figure 2c, the index records two distinct entries for paths from p_9 to j_{10} . The first entry has a depth of 2, indicating paths that cross boundaries twice (entering function g and returning to f). The second entry has a depth of 4, indicating paths that additionally cross into and out of function h (via i_{24} and c_{18}) before eventually reaching j_{10} .

Index Construction via Adaptive Depth. Computing the full transitive closure of feasible paths between all BNs is computationally intractable. To balance offline overhead and online performance, our offline construction employs an *Adaptive Depth* strategy. Instead of computing paths to a uniform maximum depth, we assign different expansion budgets to different BNs based on their structural complexity.

Consider the two parameters p_9 and q_9 . First, regarding inflow volume, more nodes can potentially reach p_9 (e.g., x_2, y_3, z_4), whereas only y_3 can reach q_9 . Second, regarding outflow search space, p_9 initiates a vastly larger search space. As analyzed earlier, crossing just two function boundaries from p_9 yields 6 candidate paths, whereas q_9 yields only 1. Therefore, it is highly cost-effective to invest offline resources into p_9 to resolve its complex branches early. As shown in Table (c), p_9 is assigned a higher budget, and the index computes its feasible paths up to depth 5, ultimately revealing that no feasible path exists to r_5 ($\emptyset$). Conversely, q_9 is assigned a minimal budget, computing paths only up to depth 1.

Accelerating Online Queries. During the online phase, the pre-computed index drastically reduces SMT overhead. For the query (y_3, r_6) , the search first reaches p_9 ($y_3 \rightarrow z_4 \rightarrow p_9$). It queries the index for a path from p_9 to the target's enclosing boundary r_5 . The index immediately returns $\emptyset$, pruning 6 SMT checks instantly. Next, the search explores q_9 . Because the precomputed index does not contain a direct entry for (q_9, r_5) (having exhausted the depth limit), the algorithm triggers *lazy expansion*. It on-the-fly composes shorter cached witnesses into a longer extension by assembling the shallow index entries: (q_9, i_{24}) , (i_{24}, k_{11}) , and (k_{11}, r_5) . It then connects this to r_6 , forming a single candidate: $y_3 \rightarrow q_9 \rightsquigarrow i_{24} \rightsquigarrow k_{11} \rightsquigarrow r_5 \rightarrow r_6$. The SMT solver is invoked *exactly once* to verify this assembled path, finding it SAT. For the query (x_2, r_6) , the flow reaches p_9 ($x_2 \rightarrow z_4 \rightarrow p_9$). The algorithm queries the index for (p_9, r_5) , which returns $\emptyset$. The search terminates immediately. The query concludes that r_6 is infeasible from x_2 with *zero* SMT invocations.

4 Approach

This section details our two-phase framework. Following the motivating example, we formalize the boundary-anchored index (§ 4.1) and an adaptive depth strategy (§ 4.2) to guide the offline index construction via compositional generation and pruning (§ 4.3). Finally, we present the online query algorithm with lazy expansion and its theoretical guarantees (§ 4.4), concluding with multi-dimensional budgets for offline scalability (§ 4.5).

4.1 Boundary-Anchored Index Structure

This subsection formalizes the *boundary-anchored index*. We define *boundary nodes* as compositional interfaces, introduce *boundary depth* to quantify inter-procedural distance, and structure the index to cache SMT-verified candidate paths across these boundaries.

Definition 4.1 (Boundary Node). Let $\mathcal{B} \subseteq V$ be the set of *boundary nodes* (BNs):

$$\mathcal{B} \triangleq \{v \in V \mid \exists u, \phi, \gamma. (u \xrightarrow{\gamma, \phi} v) \in E \wedge \gamma \neq \epsilon\}.$$

We classify BNs into two (not necessarily disjoint) subsets based on their incoming interprocedural edges: *argument nodes* ($\mathcal{B}_{\text{arg}}$), which receive flow via call edges ($\llbracket_i$), and *callsite output nodes* ($\mathcal{B}_{\text{out}}$), which receive flow via return edges ($\llbracket_i$). By construction, $\mathcal{B} = \mathcal{B}_{\text{arg}} \cup \mathcal{B}_{\text{out}}$.

Anchoring at BNs is akin to summary edges in IFDS/IDE analyses: intra-procedural regions are dense, while interprocedural connections go through sparse, compositional interfaces. Indexing only BNs cleanly separates context-tagged interface transitions ($\gamma \neq \epsilon$) from tag-free intra-procedural propagation ($\gamma = \epsilon$), enabling the two parts to be handled independently. To formalize this abstraction, we can project any concrete, complex VFG path onto a simplified sequence consisting solely of its boundary crossings.

Definition 4.2 (Boundary Node Sequence). Let $\pi : u \rightsquigarrow v$ be a path in the VFG. The *boundary node sequence* of π is defined as the ordered tuple $(b_1, b_2, \dots, b_k)$ such that $b_i \in \pi$ and $b_i \in \mathcal{B}$ for all $1 \leq i \leq k$, preserving their exact order of appearance along π .

The length of a boundary sequence naturally defines the structural distance of an inter-procedural path. By measuring distance through boundary crossings rather than raw edge counts, we establish a scalable metric for inter-procedural traversal. We formalize *boundary depth* as follows:

Definition 4.3 (Boundary Depth). For any VFG path π , its *boundary depth* is defined as the number of inter-procedural edges it contains:

$$\text{bd}(\pi) \triangleq |\text{tag}(\pi)|.$$

Intuitively, $\text{bd}(\pi)$ is the exact number of times π crosses procedure boundaries.

With the boundary depth established, we can now define the structure of our offline index. The construction of an index entry begins by identifying *realizable paths* between two BNs at a specific depth, which serve as our *candidates*. These candidates are subsequently verified for logical feasibility using an SMT solver, with the successful results cached into the index.

Definition 4.4 (Candidate Path Set). Given two boundary nodes $u, v \in \mathcal{B}$ and a depth $d \geq 1$, the candidate path set $\text{Cand}[u, v, d]$ consists of all contextually realizable paths from u to v with exactly depth d :

$$\text{Cand}[u, v, d] \triangleq \{\pi \mid \pi : u \rightsquigarrow^{\text{cs}} v \wedge \text{bd}(\pi) = d\}.$$

Definition 4.5 (Boundary-Anchored Index Entry). A *boundary-anchored index entry* is keyed by a triple (u, v, d) where $u, v \in \mathcal{B}$ and $d \geq 1$. It stores a strict partition of the candidate paths $\text{Cand}[u, v, d]$ into logically feasible (SAT) and infeasible (UNSAT) sets:

$$\mathcal{I}[u, v, d] \triangleq (\text{SAT}[u, v, d], \text{UNSAT}[u, v, d]),$$

where:

$$\begin{aligned} \text{SAT}[u, v, d] &\triangleq \{\pi \in \text{Cand}[u, v, d] \mid \pi : u \xrightarrow{\text{ps:sat}} v\}, \\ \text{UNSAT}[u, v, d] &\triangleq \{\pi \in \text{Cand}[u, v, d] \mid \pi : u \xrightarrow{\text{ps:unsat}} v\}. \end{aligned}$$

To formalize the construction of these entries, we define the SMT verification operator `VERIFY`. Given any set of candidate paths C , `VERIFY` evaluates the path conditions via an SMT solver and partitions C into feasible and infeasible subsets:

$$\text{VERIFY}(C) \triangleq (\{\pi \in C \mid \text{pc}(\pi) \text{ is SAT}\}, \{\pi \in C \mid \text{pc}(\pi) \text{ is UNSAT}\}).$$

Consequently, the index entry for a given boundary triple can be directly computed as:

$$\mathcal{I}[u, v, d] = \text{VERIFY}(\text{Cand}[u, v, d]).$$

Because the `VERIFY` operation relies on heavy SMT invocations, exhaustively applying it to all realizable paths across the program is fundamentally intractable. This prohibitive cost dictates that we selectively allocate SMT resources to compute only the most cost-effective index entries.

4.2 Adaptive Index Selection

Exhaustively computing the index up to a maximum depth $D_{\max}^{\text{idx}}$ for all boundary nodes is computationally intractable. To optimize offline index construction, we first introduce an adaptive depth strategy. Instead of applying a uniform depth limit, this strategy assigns a tailored expansion depth $\delta(v) \in [0, D_{\max}^{\text{idx}}]$ to each boundary node $v \in \mathcal{B}$.

The assigned depth is determined by a *priority score* that balances two critical factors: the likelihood of the boundary node being accessed during online queries, and the estimated size of the search space it will encompass. We compute this score on a logarithmic scale:

$$\text{Score}(v) = \log \left(\text{Acc}(v) \times \text{Space}(v, D_{\max}^{\text{idx}}) \right)$$

where $\text{Acc}(v)$ is the access probability proxy, and $\text{Space}(v, D_{\max}^{\text{idx}})$ estimates the search space volume.

Access Probability Proxy (Acc). During an online query, the initial search originates from a query source and traverses the VFG until it encounters a boundary node. Consequently, a boundary node is more likely to be visited if it acts as the first reachable boundary for a large number of non-boundary nodes. We formalize this by defining a backward reachability set restricted to non-boundary nodes.

For a given boundary node $v \in \mathcal{B}$, a backward reachability traversal collects all nodes $u \in V \setminus \mathcal{B}$ that can reach v via a path containing no boundary nodes other than v itself. Formally, let a path π from u to v be represented by its vertex sequence $u_0, u_1, \dots, u_m$, where $u_0 = u$ and $u_m = v$. We define the *backward reachability set* for v as:

$$\text{Reach}^{-1}(v) \triangleq \left\{ u \in V \setminus \mathcal{B} \mid \exists \pi = u_0 \dots u_m \text{ s.t. } u_0 = u, u_m = v, \text{ and } \forall j < m, u_j \notin \mathcal{B} \right\}.$$

The condition $\forall j < m, u_j \notin \mathcal{B}$ elegantly captures the traversal termination criterion: the path strictly originates from a non-boundary node and does not cross or start from any other boundary nodes before reaching v . Computing this set for all boundary nodes can be done efficiently in $O(|V| + |E|)$ time via a single backward graph traversal. The access probability proxy is then simply defined as the cardinality of this set:

$$\text{Acc}(v) \triangleq |\text{Reach}^{-1}(v)|.$$

Search Space Estimation (Space). To estimate the search space, we abstract the VFG into a *Boundary Index Graph*, denoted as $\mathcal{G}_B = (\mathcal{B}, \mathcal{E}_B, W)$. Here, the vertex set $\mathcal{B}$ consists of all boundary nodes. A directed edge $(u, w) \in \mathcal{E}_B$ exists if there is at least one feasible depth-1 indexed path from u to w . The weight function $W : \mathcal{E}_B \rightarrow \mathbb{N}^+$ assigns each edge the total number of such feasible

paths. Formally, letting $\text{SAT}[u, w, 1]$ denote the set of satisfiable depth-1 indexed paths from u to w , we define the edge weight as:

$$W(u, w) \triangleq |\text{SAT}[u, w, 1]|.$$

During actual online queries, path expansion is strictly constrained by extended Dyck-CFL reachability to ensure context sensitivity. That is, for a concatenated path π , its tag word must satisfy $\text{tag}(\pi) \in L_{\text{ctx}}$. However, accurately tracking the extended Dyck-reachability states (e.g., matching calls and returns) over multiple lookahead steps is computationally prohibitive for a lightweight heuristic. Therefore, to ensure efficiency, we deliberately relax the L_{ctx} constraints during this estimation, treating $\mathcal{G}_B$ as a standard directed graph where any edge sequence is considered structurally valid. This relaxation allows us to compute a fast, structural upper-bound estimation of the search space.

We then employ Dynamic Programming (DP) to estimate the search space $\text{Space}(v, k)$, which represents the total number of boundary-level paths of length up to k originating from v . Since the online search expands iteratively from one boundary to the next, estimating the search space explored from v within k steps is equivalent to counting the number of weighted paths of length up to k on $\mathcal{G}_B$. This recurrence follows the path-counting principle underlying Ball-Larus path profiling [6]; instead of profiling executed acyclic paths, we count depth-bounded paths on a weighted boundary graph as a static search-space proxy. The DP state transition is formulated as:

$$\begin{aligned} \text{Space}(v, 0) &= 1, \\ \text{Space}(v, k) &= 1 + \sum_{(v,u) \in \mathcal{E}_B} W(v, u) \cdot \text{Space}(u, k-1) \quad \text{for } k \geq 1. \end{aligned}$$

Intuitively, the base case $\text{Space}(v, 0) = 1$ represents the trivial path stopping at v . For $k \geq 1$, the search can terminate at v (contributing 1) or transition to a neighboring boundary u . Since there are $W(v, u)$ feasible indexed paths to reach u , and from u the space expands by $\text{Space}(u, k-1)$ in the remaining $k-1$ steps, the product $W(v, u) \cdot \text{Space}(u, k-1)$ captures the total extended paths routed through u .

Ultimately, $\text{Space}(v, D_{\text{max}}^{\text{idx}})$ estimates the number of paths encompassed when expanding node v to depth $D_{\text{max}}^{\text{idx}}$. Thanks to the DP formulation, this estimation process is highly efficient, computing the search space for all boundary nodes in $O(|\mathcal{E}_B| \cdot D_{\text{max}}^{\text{idx}})$ time.

Depth Mapping. After computing the logarithmic scores for all $v \in \mathcal{B}$, we apply Min-Max normalization to map the scores into the continuous range $[0, 1]$. Finally, the normalized score is scaled by $D_{\text{max}}^{\text{idx}}$ and rounded to the nearest integer to determine the adaptive depth limit $\delta(v)$. The complete procedure is summarized in Algorithm 1.

4.3 Offline Index Construction

Bounded by the computed adaptive depths, our offline phase constructs an index of feasible paths between boundary nodes via compositional generation (§ 4.3.1), optimized by monotonicity-driven pruning to minimize SMT invocations (§ 4.3.2).

4.3.1 Compositional Index Construction. We construct the offline index using a compositional strategy: iteratively generating longer candidate paths by systematically composing previously established shorter ones. Specifically, candidates of depth d are constructed by concatenating candidates of depth $d-1$ with base candidates of depth 1. To formalize this generation process and rigorously prove its correctness, we first establish the foundational operation of path concatenation. As defined in previous sections:

Algorithm 1: Adaptive Depth Strategy**Input:** Boundary nodes $\mathcal{B}$, Max index depth $D_{\max}^{\text{idx}}$, Pre-computed Index $\mathcal{I}$.**Output:** Depth mapping $\delta : \mathcal{B} \rightarrow \{0, 1, \dots, D_{\max}^{\text{idx}}\}$.

```

1 Function ComputeAdaptiveDepth( $\mathcal{B}, D_{\max}^{\text{idx}}, \mathcal{I}$ ):
2   for  $v \in \mathcal{B}$  do
3      $\text{Acc}(v) \leftarrow |\text{Reach}^{-1}(v)|$ ;
4      $\text{Space}(v, 0) \leftarrow 1, \quad \forall v \in \mathcal{B}$ ;
5      $W(v, u) \leftarrow |\mathcal{I}.\text{SAT}[v, u, 1]|, \quad \forall v, u \in \mathcal{B}$ ;
6     for  $k \leftarrow 1$  to  $D_{\max}^{\text{idx}}$  do
7       for  $v \in \mathcal{B}$  do
8          $\text{Space}(v, k) \leftarrow 1 + \sum_{u \in \mathcal{B}} W(v, u) \cdot \text{Space}(u, k - 1)$ ;
9     for  $v \in \mathcal{B}$  do
10       $\text{Score}(v) \leftarrow \log(\text{Acc}(v) \cdot \text{Space}(v, D_{\max}^{\text{idx}}))$ ;
11   $S_{\min} \leftarrow \min_{v \in \mathcal{B}} \text{Score}(v), S_{\max} \leftarrow \max_{v \in \mathcal{B}} \text{Score}(v)$ ;
12  for  $v \in \mathcal{B}$  do
13     $\hat{\text{Score}}(v) \leftarrow \frac{\text{Score}(v) - S_{\min}}{S_{\max} - S_{\min}}$ ;
14     $\delta(v) \leftarrow \lfloor \hat{\text{Score}}(v) \cdot D_{\max}^{\text{idx}} + 0.5 \rfloor$ ;
15  return  $\delta$ ;

```

Definition 4.6 (Path Concatenation). Let π_1 be a path from u to v and π_2 be a path from v to w . Their concatenation, written $\pi_1 \circ \pi_2$, is the path from u to w obtained by traversing π_1 followed by π_2 .

By Definition 4.6, the tag and boundary-depth functions respect concatenation:

$$\text{tag}(\pi_1 \circ \pi_2) = \text{tag}(\pi_1) \cdot \text{tag}(\pi_2) \quad \text{and} \quad \text{bd}(\pi_1 \circ \pi_2) = \text{bd}(\pi_1) + \text{bd}(\pi_2).$$

Consequently, by Definition 2.2, $\pi_1 \circ \pi_2$ is a realizable path iff $\text{tag}(\pi_1) \cdot \text{tag}(\pi_2) \in L_{\text{ctx}}$.

Building upon these basic properties, we abstract the composition of individual paths into a formal set-level operator. This allows us to define the multiplication of two candidate sets.

Definition 4.7 (Context-Sensitive Set Composition). Let C_1 and C_2 be two sets of paths. Their context-sensitive composition, denoted $C_1 \otimes C_2$, is defined as the set of all realizable concatenations between paths in C_1 and C_2 :

$$C_1 \otimes C_2 \triangleq \{ \pi_1 \circ \pi_2 \mid \pi_1 \in C_1, \pi_2 \in C_2, \text{tag}(\pi_1) \cdot \text{tag}(\pi_2) \in L_{\text{ctx}} \}.$$

Using this composition operator, we define the layered generation of candidates. To build candidates of depth d , we multiply candidates of depth $d - 1$ with base candidates of depth 1 across all possible intermediate boundary nodes.

Definition 4.8 (Layered Candidate Generation). For any $d \geq 2$ and boundary nodes $u, w \in \mathcal{B}$, the composed candidate set of depth d is defined as:

$$\text{CompCand}[u, w, d] \triangleq \bigcup_{v \in \mathcal{B}} (\text{Cand}[u, v, d - 1] \otimes \text{Cand}[v, w, 1]).$$

To utilize this layered composition as the foundation for our index construction algorithm, we must establish its exactness and uniqueness. Exactness ensures the composed set perfectly matches the true candidate set without omissions, while uniqueness guarantees algorithmic efficiency by preventing redundant path evaluations.

THEOREM 4.9 (EXACTNESS AND UNIQUENESS OF LAYERED GENERATION). *For all $u, w \in \mathcal{B}$ and $d \geq 2$, the layered generation is exact:*

$$\text{Cand}[u, w, d] = \text{CompCand}[u, w, d].$$

Furthermore, the generation is strictly unique: every path $\pi \in \text{Cand}[u, w, d]$ is formed by exactly one unique triple (v, π_1, π_2) such that $v \in \mathcal{B}$, $\pi_1 \in \text{Cand}[u, v, d-1]$, $\pi_2 \in \text{Cand}[v, w, 1]$, and $\pi = \pi_1 \circ \pi_2$.

PROOF. (Soundness): $\text{CompCand}[u, w, d] \subseteq \text{Cand}[u, w, d]$. Take any $\pi \in \text{CompCand}[u, w, d]$. By Definition 4.8 and Definition 4.7, there exists an intermediate node $v \in \mathcal{B}$ such that $\pi = \pi_1 \circ \pi_2$, where $\pi_1 \in \text{Cand}[u, v, d-1]$, $\pi_2 \in \text{Cand}[v, w, 1]$, and $\text{tag}(\pi_1) \cdot \text{tag}(\pi_2) \in L_{\text{ctx}}$. By Definition 4.6, $\text{tag}(\pi) = \text{tag}(\pi_1) \cdot \text{tag}(\pi_2) \in L_{\text{ctx}}$, hence π is a realizable path. Moreover, $\text{bd}(\pi) = \text{bd}(\pi_1) + \text{bd}(\pi_2) = (d-1) + 1 = d$. Therefore, by the definition of candidate sets, $\pi \in \text{Cand}[u, w, d]$.

(Completeness): $\text{Cand}[u, w, d] \subseteq \text{CompCand}[u, w, d]$. Fix any $\pi \in \text{Cand}[u, w, d]$. By definition, π is a realizable path with boundary depth $\text{bd}(\pi) = d$. According to Definition 4.2, π has a well-defined boundary node sequence $(b_0, b_1, \dots, b_d)$ with $b_0 = u$ and $b_d = w$. Let $v = b_{d-1}$ be the $(d-1)$ -th boundary node. We split π precisely at v (immediately after the $(d-1)$ -th non- ϵ edge), yielding $\pi = \pi_1 \circ \pi_2$. By this construction, $\text{bd}(\pi_1) = d-1$ and $\text{bd}(\pi_2) = 1$. Since π is realizable, $\text{tag}(\pi) \in L_{\text{ctx}}$. By the splittability property of the extended Dyck-CFL (Property 1), any split of a realizable sequence yields realizable subsequences. Thus, $\text{tag}(\pi_1) \in L_{\text{ctx}}$ and $\text{tag}(\pi_2) \in L_{\text{ctx}}$. This confirms $\pi_1 \in \text{Cand}[u, v, d-1]$ and $\pi_2 \in \text{Cand}[v, w, 1]$. Since their concatenation forms π and $\text{tag}(\pi) \in L_{\text{ctx}}$, the path π belongs to $\text{Cand}[u, v, d-1] \otimes \text{Cand}[v, w, 1]$, meaning $\pi \in \text{CompCand}[u, w, d]$.

(Uniqueness). Suppose there are two triples (v, π_1, π_2) and (v', π'_1, π'_2) forming the same path $\pi \in \text{CompCand}[u, w, d]$. Since $\text{bd}(\pi_1) = \text{bd}(\pi'_1) = d-1$, both π_1 and π'_1 must contain exactly the first $d-1$ non- ϵ edges of π . By the index structure design, the endpoints v and v' are boundary nodes, meaning they are reached immediately after crossing an interprocedural edge. Therefore, both π_1 and π'_1 must terminate exactly at the target node of the $(d-1)$ -th non- ϵ edge, which is uniquely identified as b_{d-1} in the boundary node sequence of π . This forces π_1 and π'_1 to be the exact same prefix of π , terminating at the identical node $v = v' = b_{d-1}$. Consequently, their suffixes must also be identical ($\pi_2 = \pi'_2$). Thus, the triple (v, π_1, π_2) is strictly unique. $\square$

Therefore, an algorithm that constructs $\text{Cand}[u, w, d]$ by iterating over all intermediate boundary nodes v and computing the context-sensitive composition $\text{Cand}[u, v, d-1] \otimes \text{Cand}[v, w, 1]$ will generate every target path exactly once, strictly avoiding any duplication.

4.3.2 Monotonicity-Driven Index Construction. To avoid invoking the expensive SMT solver for every generated candidate path, we leverage the path-sensitive feasibility results already cached in our index. Recall from Section 2.2 that the path condition $\text{pc}(\pi)$ is defined as the conjunction of individual edge constraints. This definition inherently implies that pc is strictly compositional: for any path π decomposed into a sequence of contiguous subpaths $\pi_1 \circ \pi_2 \circ \dots \circ \pi_k$, its accumulated path condition is simply:

$$\text{pc}(\pi) = \bigwedge_{i=1}^k \text{pc}(\pi_i).$$

Consequently, the satisfiability of path conditions exhibits a *monotonicity* property: extending a path can only restrict its solution space, never expand it. This observation forms the theoretical foundation of our pruning strategy, allowing us to safely discard paths based on localized infeasibility.

LEMMA 4.10 (SUBPATH PRUNING). *Let π be a candidate path and ρ be a contiguous subpath of π . If ρ is logically infeasible (i.e., $\text{pc}(\rho)$ is unsatisfiable), then the entire path π is logically infeasible.*

PROOF. Since ρ is a contiguous subpath, π can be factored as $\pi = \alpha \circ \rho \circ \beta$, where α and β are (possibly empty) prefix and suffix paths. Based on the compositionality of path conditions, we have:

$$\text{pc}(\pi) = \text{pc}(\alpha) \wedge \text{pc}(\rho) \wedge \text{pc}(\beta).$$

If ρ is infeasible, then $\text{pc}(\rho) \equiv \text{false}$. Therefore, the entire conjunction $\text{pc}(\pi)$ evaluates to false, rendering π infeasible regardless of the constraints in α or β . $\square$

Algorithm 2: Offline Index Construction

Input: Boundary nodes $\mathcal{B}$, Max index depth $D_{\max}^{\text{idx}}$, VFG G .

Output: Offline Index $\mathcal{I}$ containing SAT and UNSAT sets.

```

1 Function BuildOfflineIndex( $\mathcal{B}, D_{\max}^{\text{idx}}, G$ ):
2    $\mathcal{I} \leftarrow \emptyset$ ;
3   for each pair  $(u, v) \in \mathcal{B} \times \mathcal{B}$  do
4      $\text{Cand}[u, v, 1] \leftarrow \{ \pi \mid \pi : u \xrightarrow{\text{cs}} v \wedge \text{bd}(\pi) = 1 \}$ ;
5      $\mathcal{I}[u, v, 1] \leftarrow \text{VERIFY}(\text{Cand}[u, v, 1])$ ;
6    $\delta \leftarrow \text{ComputeAdaptiveDepth}(\mathcal{B}, D_{\max}^{\text{idx}}, \mathcal{I})$ ;
7   for  $d \leftarrow 2$  to  $D_{\max}^{\text{idx}}$  do
8     for each pair  $(u, w) \in \mathcal{B} \times \mathcal{B}$  such that  $\delta(u) \geq d$  do
9        $\Pi_{\text{cand}} \leftarrow \emptyset$ ;
10      for each  $v \in \mathcal{B}$  do
11         $\Pi_{\text{raw}} \leftarrow \mathcal{I}.\text{SAT}[u, v, d-1] \otimes \mathcal{I}.\text{SAT}[v, w, 1]$ ;
12         $\Pi_{\text{cand}} \leftarrow \Pi_{\text{cand}} \cup \text{FilterSuffix}(\Pi_{\text{raw}}, \mathcal{I}, d)$ ;
13       $\mathcal{I}[u, w, d] \leftarrow \text{VERIFY}(\Pi_{\text{cand}})$ ;
14   return  $\mathcal{I}$ ;

15 Function FilterSuffix( $\Pi_{\text{raw}}, \mathcal{I}, d$ ):
16    $\Pi_{\text{res}} \leftarrow \emptyset$ ;
17   for each  $\pi \in \Pi_{\text{raw}}$  do
18      $\text{is\_pruned} \leftarrow \text{false}$ ;
19     for  $p \leftarrow 1$  to  $d-1$  do
20       if  $\pi[b_p \rightsquigarrow w] \in \mathcal{I}.\text{UNSAT}[b_p, w, d-p]$  then
21          $\text{is\_pruned} \leftarrow \text{true}$ ;
22         break;
23     if  $\neg \text{is\_pruned}$  then  $\Pi_{\text{res}} \leftarrow \Pi_{\text{res}} \cup \{ \pi \}$ ;
24   return  $\Pi_{\text{res}}$ ;

```

Suffix-Only Pruning under SAT Composition. Instead of blindly generating all contextually realizable paths $\text{Cand}[u, w, d]$ and then applying SMT solvers, we can optimize the construction by composing paths that are already known to be feasible. Specifically, we replace the raw candidate composition with the context-sensitive composition of SAT sets: $\text{SAT}[u, v, d-1] \otimes \text{SAT}[v, w, 1]$.

While the prefix and depth-1 extension are individually feasible, their concatenation may still be unsatisfiable due to conflicting constraints. Since we construct paths layer by layer, any subpath strictly within the prefix is already feasible. By Lemma 4.10, any new infeasibility must involve the appended edge. This elegantly reduces subpath pruning to a *suffix-only* check: we only verify subpaths ending with the new extension, realizing the strategy without redundant overhead.

LEMMA 4.11 (SUFFIX-ONLY PRUNING SUFFICES). *Let $\pi = \pi_1 \circ \pi_2$ be a path where $\pi_1 \in \text{SAT}[u, v, d-1]$ and $\pi_2 \in \text{SAT}[v, w, 1]$, with boundary node sequence $(b_0, b_1, \dots, b_d)$. If π contains a contiguous UNSAT subpath ρ bounded by boundary nodes, then there exists an index $p \in \{1, \dots, d-1\}$ such that the suffix $\pi[b_p \rightsquigarrow w]$ is UNSAT.*

PROOF. Let ρ be an UNSAT subpath of π starting at boundary node b_p and ending at boundary node b_q , where $0 \leq p < q \leq d$. If $q < d$, then ρ must be entirely contained within the prefix π_1 (since the depth-1 extension only appends the segment from b_{d-1} to $b_d = w$). However, $\pi_1 \in \text{SAT}[u, v, d-1]$ has already been proven feasible, meaning no such UNSAT subpath ρ can exist in π_1 . Thus, we must have $q = d$, which implies $b_q = w$. Consequently, ρ is exactly the suffix $\pi[b_p \rightsquigarrow w]$. By Lemma 4.10, the unsatisfiability of this suffix strictly implies π is unsatisfiable. $\square$

Overall Construction Algorithm. Theorem 4.9 and Lemma 4.11 establish the theoretical foundation for Algorithm 2. The overall offline index construction proceeds in three logical phases:

- (1) **Base Layer Initialization ($d = 1$):** We first compute all intra-procedural boundary-to-boundary realizable paths. SMT solving is applied via `VerifyCandidatesToIndex` to evaluate their path conditions. Feasible paths are recorded in $\text{SAT}[u, v, 1]$, while infeasible ones are stored in $\text{UNSAT}[u, v, 1]$. Crucially, these depth-1 SAT paths serve a dual purpose: they act as the fundamental building blocks for iterative path composition, and their cardinalities directly provide the edge weights $W(u, v)$ for the Boundary Index Graph $\mathcal{G}_B$, which drives our DP-based search space estimation heuristic.
- (2) **Adaptive Depth Allocation:** Rather than expanding all nodes to a uniform maximum depth $D_{\max}^{\text{idx}}$, we compute an adaptive depth limit $\delta(u)$ for each boundary node $u \in \mathcal{B}$. This allocation (detailed in Section 4.2) prioritizes high-utility nodes to save offline resources.
- (3) **Inductive Extension with Pruning ($d \geq 2$):** For each subsequent layer d , an extension from a starting node u is only permitted if $\delta(u) \geq d$. We concatenate feasible prefixes with base-layer feasible paths using the $\otimes$ operator. Crucially, before invoking the SMT solver on a concatenated path π , we apply the suffix-only pruning filter. We enumerate its intermediate boundary nodes b_p and query the cached UNSAT index. If any suffix is found in UNSAT, π is immediately discarded. Only the surviving paths are passed to the SMT solver.

4.4 Online Query Algorithm

Following the offline index construction, we now present the online query phase. Given an online value-flow query (s, t) , our goal is to efficiently find a feasible path $\pi : s \rightsquigarrow t$ by maximally leveraging the pre-computed index. We achieve this through an index-guided search combined with lazy index expansion, ultimately verifying the end-to-end path feasibility using an SMT solver.

4.4.1 Index-Guided Search with Lazy Expansion. During an (s, t) query, the search explores the underlying value-flow graph locally around s and t , and leverages the index upon encountering boundary nodes to effectively “jump” across complex inter-procedural logic. If the search exhausts

the precomputed offline depth limit at a given boundary, it triggers *lazy expansion* to on-the-fly compose shorter cached witnesses into longer extensions.

Algorithm 3: Index-Guided Online Query with Lazy Expansion

Input: Value-flow query (s, t) , Max query depth $D_{\max}^{\text{qry}}$, Pre-computed Index $\mathcal{I}$

Output: A feasible witness path π or $\perp$

```

1  UNCHECK  $\leftarrow \emptyset$ ;
2  Function Query( $s, t$ ):
3     $\Pi_{\text{src}} \leftarrow \{ \pi : s \rightsquigarrow x \mid x \in \mathcal{B}, bd(\pi) = 1 \}$ ;
4     $\Pi_{\text{tgt}} \leftarrow \{ \pi : y \rightsquigarrow t \mid y \in \mathcal{B}, bd(\pi) = 1 \}$ ;
5    for each  $(\pi_{\text{src}} : s \rightsquigarrow x, \pi_{\text{tgt}} : y \rightsquigarrow t) \in \Pi_{\text{src}} \times \Pi_{\text{tgt}}$  do
6      for  $d \leftarrow 1$  to  $D_{\max}^{\text{qry}}$  do
7         $\Pi_{\text{idx}} \leftarrow \text{LazyExpand}(x, y, d, \mathcal{I})$ ;
8        for each  $\pi_{\text{full}} \in \{\pi_{\text{src}}\} \otimes \Pi_{\text{idx}} \otimes \{\pi_{\text{tgt}}\}$  do
9          if  $\text{pc}(\pi_{\text{full}})$  is SAT then return  $\pi_{\text{full}}$ ;
10   return  $\perp$ ;
11 Function LazyExpand( $x, y, d, \mathcal{I}$ ):
12   if  $\mathcal{I}.\text{SAT}[x, y, d]$  exists or UNCHECK[ $x, y, d$ ] exists then
13     return  $\mathcal{I}.\text{SAT}[x, y, d] \cup \text{UNCHECK}[x, y, d]$ 
14    $\Pi_{\text{res}} \leftarrow \emptyset$ ;
15   for each  $v \in \mathcal{B}$  do
16      $\Pi_{\text{prev}} \leftarrow \mathcal{I}.\text{SAT}[x, v, d-1] \cup \text{UNCHECK}[x, v, d-1]$ ;
17      $\Pi_{\text{res}} \leftarrow \Pi_{\text{res}} \cup \text{FilterSuffix}(\Pi_{\text{prev}} \otimes \mathcal{I}.\text{SAT}[v, y, 1], \mathcal{I}, d)$ ;
18   UNCHECK[ $x, y, d$ ]  $\leftarrow \Pi_{\text{res}}$ ;
19   return  $\Pi_{\text{res}}$ ;

```

Local Search to Boundaries. The search begins by exploring the local intra-procedural paths from the source s to its reachable boundary nodes, collecting prefixes $\pi_{\text{src}} : s \rightsquigarrow x$. Crucially, these paths contain no other boundary nodes except the terminal node x (i.e., x is the first boundary node encountered). Similarly, we identify the target boundary nodes y that enclose t without any intermediate boundary nodes, collecting local suffixes $\pi_{\text{tgt}} : y \rightsquigarrow t$. The core task then reduces to finding an inter-boundary summary path $\pi_{\text{idx}} : x \rightsquigarrow y$ to bridge them.

Lazy Expansion and Path Assembly. To construct π_{idx} , the algorithm iterates through depths from 1 to $D_{\max}^{\text{qry}}$ and invokes the LazyExpand function. This iterative deepening process, combined with recursively fetching prefixes, is essentially equivalent to a DFS over the boundary index graph. LazyExpand acts as a unified interface for retrieving candidate paths between boundaries. If the requested path set at depth d was already verified offline (SAT) or generated but unverified (UNCHECK), it returns their union immediately. Here, the UNCHECK set represents candidate paths that have been structurally assembled but bypassed SMT verification due to the lazy expansion or offline budget constraints (as detailed in § 4.5). Thus, all potentially feasible candidates within $\text{Cand}[x, y, d]$ must reside in either the $\text{SAT}[x, y, d]$ or $\text{UNCHECK}[x, y, d]$ set. Therefore, LazyExpand returns their union to guarantee that no feasible paths are missed.

If the candidates are missing, LazyExpand triggers the on-the-fly expansion. It recursively fetches the realizable prefixes Π_{prev} at depth $d - 1$ and composes them with the depth-1 SAT paths. When selecting Π_{prev} , we take the union of SAT and UNCHECK sets to maintain completeness. This composition works in tandem with the FilterSuffix function to achieve subpath pruning (Lemma 4.10). Because the paths are constructed inductively layer-by-layer, any newly introduced infeasibility can be captured by a suffix-only check (Lemma 4.11). This guarantees that paths containing known UNSAT subpaths from the index are safely and completely filtered out from the UNCHECK set. The surviving paths are then cached into $\text{UNCHECK}[x, y, d]$ and returned.

Final Verification and Witness-First Termination. Once the summary path set Π_{idx} is retrieved, we directly assemble the end-to-end paths by iterating over $\pi_{full} \in \{\pi_{src}\} \otimes \Pi_{idx} \otimes \{\pi_{tgt}\}$. For each assembled full path π_{full} , we invoke the SMT_Check function. If the solver returns SAT, π_{full} is a feasible witness, and the query terminates immediately. If it returns UNSAT, the path is discarded, and the search continues.

4.4.2 Theoretical Guarantees. The integration of offline indexing and online lazy expansion maintains strict theoretical guarantees regarding soundness and completeness.

THEOREM 4.12 (WITNESS SOUNDNESS). *If the online query algorithm returns a path π for a query (s, t) , then π is a feasible path.*

PROOF. The context-sensitive set composition operator ($\otimes$) strictly enforces that all concatenated paths belong to the extended Dyck-CFL grammar, ensuring they are *realizable* (Definition 2.2). Furthermore, the algorithm reports a witness π *only* if the final explicit SMT_Check on its accumulated path constraint $\text{pc}(\pi)$ yields SAT. Thus, by Definition 2.3, the returned witness is strictly a *feasible* path. The soundness is ultimately bounded by the underlying precision of the SMT solver. $\square$

THEOREM 4.13 (BOUNDED RELATIVE COMPLETENESS). *Given a maximum expansion depth limit $D_{\max}^{qry}$, if a feasible path π exists within this bounded search space, our framework will never erroneously eliminate it. The offline pruning and online lazy expansion do not introduce false negatives up to depth $D_{\max}^{qry}$.*

PROOF. A candidate path is discarded during offline construction or online filtering if and only if it contains a proven UNSAT suffix. By Lemma 4.10, discarding such candidates is strictly safe, as any path extending an unsatisfiable subpath is inherently unsatisfiable.

To ensure no feasible path is missed, all other paths are systematically preserved. The context-sensitive composition ($\otimes$) exhaustively enumerates all realizable concatenations without omission. Paths that exceed the offline budget are retained in the UNCHECK set, and those yielding SMT unknown are conservatively treated as SAT. Consequently, the union of SAT and UNCHECK forms a lossless frontier of the search space up to depth $D_{\max}^{qry}$. During the online query, the algorithm systematically evaluates this frontier on-demand, guaranteeing that if a feasible witness exists within $D_{\max}^{qry}$, it will be eventually discovered. $\square$

4.5 Bounding Offline Construction via Multi-Dimensional Budgets

In the worst case, the number of candidate paths grows exponentially with the composition depth d . To guarantee offline efficiency and prevent state explosion, we introduce a multi-dimensional budget system to strictly bound the index construction:

- **Global Depth Budget:** To constrain the overall offline overhead, we impose a global upper bound on the cumulative adaptive depth across all boundary nodes (i.e., $\sum \delta(v)$). When distributing this limited budget, we prioritize boundary nodes that are heuristically assigned deeper desired depths. This ensures that computational resources are strictly concentrated on

thoroughly indexing the most critical interprocedural transitions, rather than being diluted across trivial nodes.

- **SMT Verification Budget:** During the candidate path verification step ($\text{VERIFY}(\Pi_{\text{cand}})$ in Algorithm 2), we limit the number of SMT solver invocations per index entry. Once this budget is exhausted, any remaining candidate paths bypass SMT evaluation and are directly routed to the UNCHECKED set.
- **Composition Budget:** For each layer in Algorithm 2, we impose a strict limit on the number of intermediate boundary nodes v used to compose new paths. If this budget is triggered, the remaining intermediate nodes are still used for composition to prevent path loss, but the resulting concatenated paths skip SMT verification and are directly recorded into the UNCHECK set. This prevents localized path explosion within highly complex functions.

To maintain bounded relative completeness (Theorem 4.13) and soundness (Theorem 4.12), the offline phase ensures a *lossless frontier*. When a budget exhausts, unverified candidates are never discarded; they are deferred to the UNCHECK set and remain available for subsequent compositions. This guarantees all potentially feasible paths up to depth $\delta(u)$ are strictly preserved. Ultimately, these budgets merely tune the offline-online trade-off: higher offline budgets resolve more paths to prune the online search space, while tighter budgets shift the SMT verification workload to the online phase.

5 Implementation

We implemented a prototype of SPONGE to evaluate its practical effectiveness, focusing on its core infrastructure, parallel indexing, and query optimizations.

Base Infrastructure. We implemented SPONGE in C++ on the Pinpoint framework [43], using LLVM [25] for C/C++ analysis. Our VFG construction follows standard “soundy” practices [2, 28, 43, 45] for scalability: we unroll loops twice, bypass inline assembly, and manually model standard libraries. Indirect calls are resolved via flow-insensitive pointer analysis [61]. Following common scalable analysis practices [2, 43, 52], we assume function pointer parameters do not alias.

Offline Indexing. We accelerate index construction via layer-wise parallelization (Algorithm 2). Inter-layer transitions are sequential to let depth $d - 1$ prune unsatisfiable subpaths at depth d , while intra-layer exploration and SMT checks are fully parallelized (unlike Pinpoint’s bottom-up approach [43]). For fast online queries, the index resides entirely in main memory, serialized via Cap’n Proto [33] to decouple the offline and online phases.

Path Sensitivity. We use Z3 [15] for SMT solving. To scale path-sensitive analysis, we adopt a *no-caching* strategy [44]: rather than storing explicit path conditions in the index, SPONGE dynamically recovers them from the VFG during verification. This accelerates solving and significantly reduces memory pressure. For fairness, we applied this identical optimization to all baselines.

6 Evaluation

To evaluate the effectiveness and efficiency of SPONGE, we designed our experiments to answer the following Research Questions (RQs):

- **RQ1 (Online Query Performance):** How does SPONGE compare to baseline approaches in terms of query response time, timeout rates, and precision?
- **RQ2 (Offline Indexing Cost):** What is the overhead of constructing the boundary-anchored index, and how quickly is this cost amortized during online querying?
- **RQ3 (Ablation Study):** How do the adaptive-depth strategy and different depth limits affect both indexing overhead and query performance?

6.1 Experimental Setup

Datasets and Query Generation. We evaluated on 9 C/C++ projects from the SPEC2017 benchmark [7]. We constructed two datasets: *NPD* (null pointer dereference) for practical bug detection, and *Random* source-sink pairs to ensure algorithmic generality.

Since trivial queries are easily resolved by existing Dyck-CFL indexes [41], we deliberately focus on the challenging long-tail: queries guaranteed to be context-sensitively *realizable* but requiring deep path-sensitive reasoning for *feasibility*. Using Pinpoint as an oracle, we generated a pool of realizable queries crossing ≥ 2 procedural boundaries. For each project, we sampled 200 distinct queries (100 feasible, 100 infeasible under Pinpoint). To maximize difficulty while preserving the natural distribution of edge cases, we proportionally sampled queries that triggered timeouts in a standard push-down DFS, padding the remainder with the slowest non-timeout DFS queries. This yields 1,800 hard queries per dataset.

Baselines and Environment. Experiments ran on an Ubuntu 20.04 server (128-core Intel Xeon 2.60 GHz, 503 GB RAM). We compare SPONGE (index depth $d = 15$) with two baselines representing the alternatives it seeks to bridge. *Pinpoint* [43] represents whole-program, path-sensitive value-flow analysis with eager summary computation. *Naive DFS* is the unindexed on-demand baseline: it performs push-down DFS over the same guarded VFG, enforces the same extended Dyck-CFL realizability, and applies the same SMT verification to complete candidate paths, but does not consult the boundary-anchored index. Comparison with Naive DFS therefore isolates the benefit of indexed reuse, while Pinpoint provides a comparison with whole-program summarization. We evaluate Pinpoint on a 10% uniform sample because its exhaustive cost prevents evaluation on the complete workload.

We restrict direct comparisons to methods that answer the same pointer-aware source-sink value-flow queries and return feasibility-checked witnesses. Other demand-driven push-down analyses solve domain-specific data-flow problems; Boomerang, for example, targets alias queries [46, 47], while Corral [24] targets control-location reachability modulo theories. Adapting either paradigm would require changing the query encoding and recovering value-flow witnesses, confounding the effect of indexing with differences in representation and query semantics. Section 7 discusses their methodological relationships with SPONGE.

Offline preprocessing and Pinpoint used 64 threads. To bound state explosion, SPONGE enforces strict budgets: global depth (15,000), SMT verifications (100), and compositions (5, reduced to 3 for projects exceeding 1 million LoC). Online queries have a 300 s timeout and terminate upon finding the first feasible path. Search depth is capped at 20. Alongside standard intra-procedural limits (500 paths per function), this deliberately exposes our approach to a larger search space than Pinpoint for fairness.

SMT solvers are utilized in three stages: (1) Lightweight pruning by Pinpoint summaries and SPONGE's $d = 1$ base index (excluded from reported SMT statistics). (2) Standard path verification (2 s timeout) for both offline inter-procedural index and online candidates; our reported "SMT checks" strictly count these. (3) A rigorous post-hoc oracle (20 s timeout) verifies all reported witnesses to establish absolute correctness. Query outcomes are thus classified as: Timeout (TO), No Witness, False Positive (rejected by oracle), or True Positive (verified by oracle).

6.2 RQ1: Online Query Performance

To answer RQ1, we first compare SPONGE with Naive DFS on the full datasets to isolate the effect of indexed reuse. We then include Pinpoint on the 10% sampled subset to contrast SPONGE with whole-program summarization.

Table 1. Online query performance. P50 and P95 represent the median and 95th percentile query response times, including timeouts (capped at 300 s). *Online SMT* exclusively counts the expensive online candidate verifications for non-timeout queries. *Report (%)* denotes the percentage of queries that return a witness, and *Prec (%)* is the precision of these reported witnesses as confirmed by our strict independent post-hoc SMT oracle.

Dataset	Method	P50 (s)	P95 (s)	Online SMT	TO (%)	Report (%)	Prec (%)
NPD	Naive DFS	2.21	269.82	98038	4.90	60.68	90.44
	SPONGE $d = 15$	1.34	48.97	37264	1.86	61.69	90.68
Random	Naive DFS	1.65	132.27	65861	1.94	54.06	70.50
	SPONGE $d = 15$	0.75	40.13	9081	0.44	55.00	71.62

Effectiveness on the Full Dataset. Table 1 compares the online performance of SPONGE and Naive DFS. While both achieve fast median (P50) times, SPONGE heavily outperforms on the challenging long tail. On NPD, Naive DFS struggles with a P95 of 269.82 s and a 4.90% timeout (TO) rate, whereas SPONGE drastically reduces P95 to 48.97 s and TO to 1.86% (with similar proportional gains on Random, dropping P95 from 132.27 s to 40.13 s and TO from 1.94% to 0.44%). This efficiency stems from our offline index, which prunes infeasible paths early, reducing expensive online SMT checks by nearly 62% on NPD and over 86% on Random. By avoiding timeouts on complex queries, SPONGE successfully discovers valid witnesses missed by Naive DFS, simultaneously yielding higher report rates and precision across both datasets.

Table 2. Online query performance on the 10% sampled subset to compare with Pinpoint. Metric definitions follow Table 1.

Dataset	Method	P50 (s)	P95 (s)	TO (%)	Report (%)	Prec (%)
NPD	Pinpoint	> 300	> 300	56.18	39.89	90.14
	Naive DFS	2.47	263.40	5.06	60.11	88.79
	SPONGE $d = 15$	1.04	39.41	1.04	61.80	89.09
Random	Pinpoint	> 300	> 300	53.33	41.11	74.32
	Naive DFS	1.16	152.76	2.78	51.67	69.89
	SPONGE $d = 15$	0.63	38.45	0.00	54.44	70.41

Comparison with Whole-Program Analysis. Table 2 highlights the limitations of whole-program analyzers for ad hoc queries. Pinpoint’s reliance on exhaustive bottom-up tabulation pushes both its P50 and P95 response times beyond 300 s, causing timeouts on 53% to 56% of queries and dropping its report rate to around 40%–41% (SMT statistics are omitted due to architectural incompatibility). In contrast, SPONGE bridges the gap between exhaustive summarization and memoryless search. SPONGE maintains P50 times around 1 s or less and caps P95 strictly below 40 s. Consequently, it achieves near-zero timeouts (dropping to 0.00% on Random) and the highest report rates among all baselines.

Figure 3 shows cactus plots of query resolution times on the 10% sampled datasets. Pinpoint scales poorly and times out frequently. SPONGE and Naive DFS perform similarly on simple queries but diverge significantly on the challenging long tail. For complex queries, SPONGE achieves substantial speedups (often 5–10 $\times$), as shown by the log-scaled y-axis. We plot the 10% subsets to include Pinpoint; full dataset plots show nearly identical trends and are omitted for brevity.

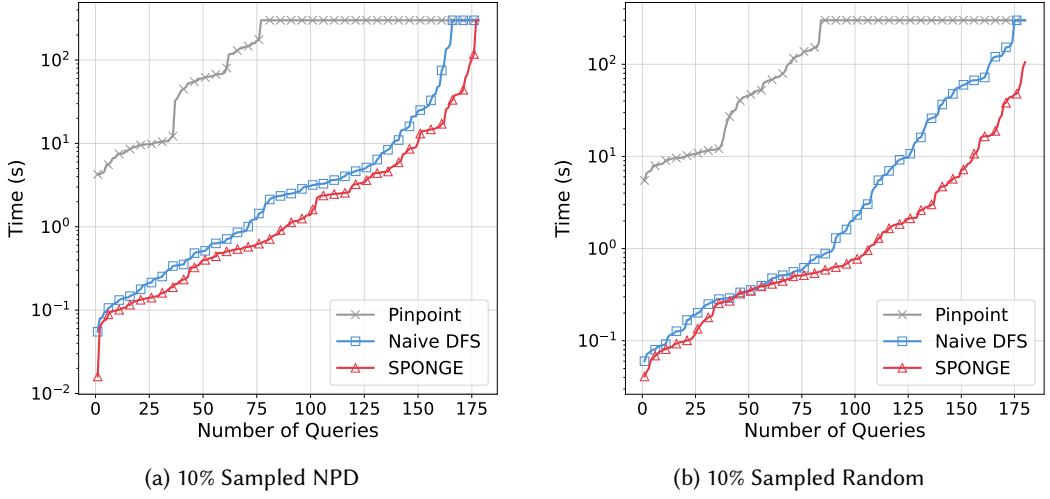


Fig. 3. Cactus plots showing the number of queries resolved (x-axis) within a given per-query time budget (y-axis, log scale) on the 10% sampled subsets. SPONGE consistently solves more queries in less time compared to the baselines.

Table 3. Offline indexing time (minutes) and break-even (BE) points. BE is derived from the average query response times (seconds) of Naive DFS and SPONGE (*Spg*). Timeouts are capped at 300 s.

Project	KLOC	Index (m)	NPD Queries			Random Queries		
			DFS (s)	Spg (s)	BE	DFS (s)	Spg (s)	BE
leela	71	231.54	0.90	0.72	75135	16.63	0.93	885
pop2	92	97.32	22.99	2.05	279	72.00	10.59	97
cam4	96	83.34	19.78	1.73	277	60.91	16.45	112
wrf	117	82.33	18.46	0.78	279	43.11	12.78	163
povray	307	232.97	7.33	5.33	6974	0.25	0.22	438188
omnetpp	591	54.81	18.93	5.85	251	4.73	2.25	1329
xalan	1412	214.96	34.55	32.02	5101	11.65	4.15	1720
parest	2962	183.51	99.22	51.79	232	11.59	9.12	4453
blender	3841	339.00	9.32	8.69	32347	2.78	2.45	62585

6.3 RQ2: Offline Indexing Cost

To answer RQ2, we analyze the offline indexing time and the break-even (BE) point. The BE point represents the number of online queries required to amortize the initial offline construction cost. When computing averages, timeouts are conservatively capped at 300 seconds. Table 3 details these metrics across the evaluated projects.

Amortization on Complex vs. Simple Queries. The cost-effectiveness of our index is particularly striking for complex queries. For projects where Naive DFS is slow (e.g., *pop2*, *cam4*, *wrf*), SPONGE achieves substantial online speedups, rapidly amortizing offline costs with break-even (BE) points well under 300 queries (e.g., fewer than 100 for *pop2* on Random). Conversely, for simple queries quickly resolved by Naive DFS (e.g., *leela*, *povray*), the performance gap narrows, increasing BE points. However, SPONGE’s primary objective is tackling challenging, long-tail paths

where traditional methods time out, rather than optimizing already-trivial queries. Therefore, shared analysis services that receive a sufficient volume of complex queries can amortize the index construction cost across clients.

Generality and Bounded Indexing Budgets. Table 3 shows offline indexing time is not strictly tied to project size: *leela* (71 KLOC) takes 232 min, while *omnetpp* (591 KLOC) takes 55 min. Instead of scaling unboundedly with total LoC, offline exploration is strictly capped by our multi-dimensional budgets (Section 4.5). Upon reaching these uniform limits, the analysis defers unresolved paths to the online phase as UNCHECK, effectively preventing state explosion regardless of project scale. SPONGE is a general framework that does not pre-assume project characteristics or query difficulty.

6.4 RQ3: Ablation Study on Indexing Strategies

To answer RQ3, we investigate the impact of our adaptive depth strategy and the chosen maximum depth limit on both offline construction overhead and online query efficiency. We focus our ablation study on three representative projects (pop2, cam4, and wrf) from the NPD dataset.

Table 4. Comparison of Uniform vs. Adaptive Depth ($d = 15$) on pop2, cam4, and wrf (NPD Dataset). *Offline SMT* refers to solver calls during index construction, whereas *Online SMT* denotes solver calls during query resolution for non-timeout queries (report rates are identical and omitted).

Project	Strategy	Offline Cost			Online Query Performance		
		Time (m)	# Entries	Offline SMT	P95 (s)	Online SMT	Precision (%)
pop2	UNIFORM $d = 15$	365.07	210501	180696	2.40	712	83.33
	SPONGE $d = 15$	97.32	86587	52342	2.41	751	84.85
cam4	UNIFORM $d = 15$	336.39	213771	183640	2.82	895	97.32
	SPONGE $d = 15$	83.34	91302	56249	2.93	871	97.32
wrf	UNIFORM $d = 15$	306.86	171661	145669	2.62	578	87.60
	SPONGE $d = 15$	82.33	82479	51319	2.60	635	89.92

Impact of the Adaptive Depth Strategy. We compare SPONGE’s adaptive depth strategy against a uniform depth baseline ($d = 15$). As Table 4 shows, blindly exploring all paths (uniform) incurs over 300 minutes of offline computation, yields up to 213,000 index entries per project, and requires up to 183,000 offline SMT calls. By intelligently allocating depth budgets based on node criticality, SPONGE reduces offline time by nearly 75%, cuts offline SMT calls by roughly 65% to 71%, and reduces storage overhead (index entries) by more than half. Crucially, this massive offline reduction preserves online efficiency: P95 response times remain strictly under 3 s, and both online SMT calls and precision remain virtually unchanged, proving SPONGE successfully isolates the most reusable value-flow segments.

Impact of Index Depth Limits. Next, we evaluate maximum depth limits ($d \in \{1, 5, 10, 15\}$) using the adaptive strategy (Table 5). A shallow index ($d = 1$) builds quickly (in about 2 minutes) but fails to prune the online search space effectively, causing high online SMT checks, P95 latencies exceeding 185 s, and 4.0%–4.5% timeout rates. Increasing d naturally raises offline costs but yields substantial online dividends. At $d = 15$, the index captures deep inter-procedural constraints, eliminating timeouts entirely, reducing online SMT checks by over 90% compared to $d = 1$, and dropping P95 latencies strictly under 3 s. This inverse relationship between offline cost and online latency highlights the necessity of deep indexing.

Table 5. Performance Trade-offs across Different Index Depths using SPONGE on pop2, cam4, and wrf (NPD Dataset). Metric definitions follow Table 4.

Project	Depth	Time (m)	# Entries	P95 (s)	TO (%)	Online SMT
pop2	$d = 1$	2.2	37979	195.19	4.50	8130
	$d = 5$	41.76	58387	16.08	0.00	4072
	$d = 10$	62.67	68321	14.32	0.00	2420
	$d = 15$	97.32	86587	2.41	0.00	751
cam4	$d = 1$	2.3	38823	192.72	4.00	14124
	$d = 5$	35.43	55837	21.55	0.00	6608
	$d = 10$	79.50	71279	13.34	0.00	3424
	$d = 15$	83.34	91302	2.93	0.00	871
wrf	$d = 1$	2.13	34049	185.44	4.00	8105
	$d = 5$	15.57	43634	12.92	0.00	4798
	$d = 10$	35.07	57594	7.51	0.00	2151
	$d = 15$	82.33	82479	2.60	0.00	635

6.5 Limitations and Future Work

While our boundary-anchored index significantly accelerates complex value-flow queries, it introduces inherent memory and lookup overheads. At $d = 15$, the loaded index consumes 1.06 GB on average across the nine projects, ranging from 102 MB to 4.10 GB. Because the index resides entirely in memory, the current implementation risks out-of-memory errors on exceptionally large codebases. Offloading the index to a persistent database would alleviate memory pressure but inevitably increase lookup latency [56, 58]. Consequently, for trivially simple queries where the search space is minimal, the retrieval overhead can outweigh the benefits and make a naive search marginally faster.

Additionally, exhaustively indexing dense clusters of purely satisfiable paths inflates the index size without yielding proportional performance gains, which is a localized manifestation of the classic path explosion problem [8, 40]. Our framework also currently treats ad hoc queries as strictly independent events, whereas real-world analysts typically issue sequences of highly correlated queries during interactive auditing [4, 29]. To address these limitations, future work will transition our static precomputation model into a dynamic system-level cache. We plan to introduce on-the-fly index augmentation, cache management strategies to retain query hotspots and evict cold entries, and adaptive solver strictness. These enhancements will make the framework more resilient to path explosion and better optimized for continuous interactive auditing. Finally, our evaluation assumes a stable program revision between index rebuilds. Incremental index maintenance under code evolution is orthogonal to the planned-versus-ad-hoc query distinction and remains future work.

7 Related Work

We situate our work at the intersection of high-precision value-flow analysis, demand-driven reachability, and ad hoc program indexing.

High-Precision Value-Flow Analysis. Classical interprocedural frameworks such as IFDS/IDE exploit procedure summaries as compositional interfaces [36, 39]. Whole-program analyzers such as ESP, Saturn, Calysto, and Pinpoint use procedure summaries and path-sensitive reasoning to

detect predefined properties [2, 5, 14, 43]. Subsequent value-flow systems reduce this cost through summary sharing among multiple predefined checkers [42], bottom-up parallelization [45, 52], or persistent library summaries [57]. These summaries encode transfer or feasibility information for predefined analysis domains and properties. SPONGE likewise uses procedure boundaries as compositional interfaces, but indexes concrete value-flow segments and their SMT-verified feasibility results independently of future source–sink endpoints, enabling reuse across unrelated ad hoc queries.

Demand-Driven and Push-Down Analysis. Demand-driven interprocedural analyses answer a given query by restricting exploration to relevant program regions. Classical formulations use CFL reachability or weighted push-down systems to preserve calling contexts [18, 37]; Boomerang applies this principle to context-, flow-, and field-sensitive alias queries [46, 47]. Goal-directed verification exploits procedural structure in a different setting. Corral checks a specified control-location reachability property modulo theories and uses stratified inlining to expose procedures as needed [24], while program transformation can lift deeply nested assertions toward the entry procedure to accelerate such checking [23]. Both forms of analysis are guided by a query or verification goal already known at analysis time. In contrast, SPONGE precomputes selected segments and their SMT-verified feasibility on a pointer-aware value-flow graph before future source–sink pairs are known. Its adaptive-depth policy concentrates this query-independent effort at boundaries with high estimated access and search-space costs. Memoized symbolic execution also reuses solver results, but populates its cache reactively and thus retains cold-start costs for independent queries [59].

Ad Hoc Querying and Indexing. To support interactive vulnerability auditing, frameworks like CodeQL [4, 11] and Joern [58] decouple offline extraction of program representations (e.g., relational facts or property graphs) from online querying, allowing analysts to search for unforeseen code patterns. Reachability indices similarly accelerate online queries, from general graph indices [1, 12, 19] to Dyck-CFL indices [9, 20, 61], extended Dyck-CFL reachability for source–sink pairs [41], and compositional call-graph indices [55]. These representations and indices capture structural reachability, whereas SPONGE records path-feasibility outcomes that can directly prune SMT-based value-flow queries.

8 Conclusion

We presented SPONGE, a two-phase framework that accelerates high-precision ad hoc value-flow queries. To bridge the gap between intractable offline tabulation and memoryless online search, SPONGE introduces an adaptive boundary-anchored index. By selectively precomputing feasible value-flow segments at critical procedure boundaries and stitching them on the fly during an index-guided push-down search, SPONGE effectively eliminates repetitive state exploration and redundant SMT invocations. Evaluations on large-scale C/C++ projects demonstrate that this approach drops 95th-percentile query latency from approximately 270 s to under 50 s and nearly eliminates timeouts, making complex on-demand analysis practical for human auditors and AI agents.

Data-Availability Statement

A binary reproduction artifact for SPONGE is publicly available on Zenodo [31]. The complete source code is not publicly available because SPONGE depends on non-public components of Pinpoint that cannot be redistributed.

Acknowledgments

We thank the anonymous reviewers for their constructive feedback, which helped improve the clarity and presentation of this paper.

References

- [1] Rakesh Agrawal, Alexander Borgida, and H. V. Jagadish. 1989. Efficient Management of Transitive Relationships in Large Data and Knowledge Bases. In *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data, Portland, Oregon, USA, May 31 - June 2, 1989*, James Clifford, Bruce G. Lindsay, and David Maier (Eds.). ACM Press, 253–262. doi:10.1145/67544.66950
- [2] Alex Aiken, Suhabe Bugrara, Isil Dillig, Thomas Dillig, Brian Hackett, and Peter Hawkins. 2007. An Overview of the Saturn Project. In *Proceedings of the 7th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering, PASTE'07, San Diego, California, USA, June 13-14, 2007*, Manuvir Das and Dan Grossman (Eds.). ACM, 43–48. doi:10.1145/1251535.1251543
- [3] Thanassis Avgerinos, Alexandre Rebert, Sang Kil Cha, and David Brumley. 2014. Enhancing Symbolic Execution with Veritesting. In *36th International Conference on Software Engineering, ICSE '14, Hyderabad, India - May 31 - June 07, 2014*, Pankaj Jalote, Lionel C. Briand, and André van der Hoek (Eds.). ACM, 1083–1094. doi:10.1145/2568225.2568293
- [4] Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. 2016. QL: Object-oriented Queries on Relational Data. In *30th European Conference on Object-Oriented Programming, ECOOP 2016, Rome, Italy, July 18-22, 2016 (LIPICs, Vol. 56)*, Shriram Krishnamurthi and Benjamin S. Lerner (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2:1–2:25. doi:10.4230/LIPICs.ECOOP.2016.2
- [5] Domagoj Babic and Alan J. Hu. 2008. Calysto: Scalable and Precise Extended Static Checking. In *30th International Conference on Software Engineering (ICSE 2008), Leipzig, Germany, May 10-18, 2008*, Wilhelm Schäfer, Matthew B. Dwyer, and Volker Gruhn (Eds.). ACM, 211–220. doi:10.1145/1368088.1368118
- [6] Thomas Ball and James R. Larus. 1996. Efficient Path Profiling. In *Proceedings of the 29th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 29, Paris, France, December 2-4, 1996*, Stephen W. Melvin and Steve Beaty (Eds.). ACM/IEEE Computer Society, 46–57. doi:10.1109/MICRO.1996.566449
- [7] James Bucek, Klaus-Dieter Lange, and Joakim von Kistowski. 2018. SPEC CPU2017: Next-Generation Compute Benchmark. In *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering, ICPE 2018, Berlin, Germany, April 09-13, 2018*, Katinka Wolter, William J. Knottenbelt, André van Hoorn, and Manoj Nambiar (Eds.). ACM, 41–42. doi:10.1145/3185768.3185771
- [8] Cristian Cadar and Koushik Sen. 2013. Symbolic Execution for Software Testing: Three Decades Later. *Commun. ACM* 56, 2 (2013), 82–90. doi:10.1145/2408776.2408795
- [9] Krishnendu Chatterjee, Bhavya Choudhary, and Andreas Pavlogiannis. 2018. Optimal Dyck Reachability for Data-Dependence and Alias Analysis. *Proc. ACM Program. Lang.* 2, POPL (2018), 30:1–30:30. doi:10.1145/3158118
- [10] Sigmund Cherem, Lonnie Princehouse, and Radu Rugina. 2007. Practical Memory Leak Detection Using Guarded Value-Flow Analysis. In *Proceedings of the ACM SIGPLAN 2007 Conference on Programming Language Design and Implementation, San Diego, California, USA, June 10-13, 2007*, Jeanne Ferrante and Kathryn S. McKinley (Eds.). ACM, 480–491. doi:10.1145/1250734.1250789
- [11] CodeQL. 2026. CodeQL. <https://codeql.github.com/>.
- [12] Edith Cohen, Eran Halperin, Haim Kaplan, and Uri Zwick. 2003. Reachability and Distance Queries via 2-Hop Labels. *SIAM J. Comput.* 32, 5 (2003), 1338–1355. doi:10.1137/S0097539702403098
- [13] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck. 1989. An Efficient Method of Computing Static Single Assignment Form. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '89)*, Association for Computing Machinery, New York, NY, USA, 25–35. doi:10.1145/75277.75280
- [14] Manuvir Das, Sorin Lerner, and Mark Seigle. 2002. ESP: Path-Sensitive Program Verification in Polynomial Time. In *Proceedings of the 2002 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), Berlin, Germany, June 17-19, 2002*, Jens Knoop and Laurie J. Hendren (Eds.). ACM, 57–68. doi:10.1145/512529.512538
- [15] Leonardo Mendonça de Moura and Nikolaj S. Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings (Lecture Notes in Computer Science, Vol. 4963)*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer, 337–340. doi:10.1007/978-3-540-78800-3_24
- [16] Lisa Nguyen Quang Do and Eric Bodden. 2022. Explaining Static Analysis With Rule Graphs. *IEEE Trans. Software Eng.* 48, 2 (2022), 678–690. doi:10.1109/TSE.2020.2999534
- [17] Seongjoon Hong, Junhee Lee, Jeongsoo Lee, and Hakjoo Oh. 2020. SAVER: Scalable, Precise, and Safe Memory-Error Repair. In *ICSE '20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June - 19 July, 2020*,

- Gregg Rothermel and Doo-Hwan Bae (Eds.). ACM, 271–283. doi:10.1145/3377811.3380323
- [18] Susan Horwitz, Thomas W. Reps, and Shmuel Sagiv. 1995. Demand Interprocedural Dataflow Analysis. In *Proceedings of the Third ACM SIGSOFT Symposium on Foundations of Software Engineering, SIGSOFT 1995, Washington, DC, USA, October 10-13, 1995*, Gail E. Kaiser (Ed.). ACM, 104–115. doi:10.1145/222124.222146
- [19] H. V. Jagadish. 1990. A Compression Technique to Materialize Transitive Closure. *ACM Trans. Database Syst.* 15, 4 (1990), 558–598. doi:10.1145/99935.99944
- [20] Adam Husted Kjelstrøm and Andreas Pavlogiannis. 2022. The Decidability and Complexity of Interleaved Bidirected Dyck Reachability. *Proc. ACM Program. Lang.* 6, POPL (2022), 1–26. doi:10.1145/3498673
- [21] John Kodumal and Alex Aiken. 2004. The Set Constraint/CFL Reachability Connection in Practice. In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation 2004, Washington, DC, USA, June 9-11, 2004*, William W. Pugh and Craig Chambers (Eds.). ACM, 207–218. doi:10.1145/996841.996867
- [22] Daniel Kroening and Michael Tautschnig. 2014. CBMC - C Bounded Model Checker - (Competition Contribution). In *Tools and Algorithms for the Construction and Analysis of Systems - 20th International Conference, TACAS 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5-13, 2014. Proceedings (Lecture Notes in Computer Science, Vol. 8413)*, Erika Ábrahám and Klaus Havelund (Eds.). Springer, 389–391. doi:10.1007/978-3-642-54862-8_26
- [23] Akash Lal and Shaz Qadeer. 2014. A Program Transformation for Faster Goal-Directed Search. In *Formal Methods in Computer-Aided Design, FMCAD 2014, Lausanne, Switzerland, October 21-24, 2014*. IEEE, 147–154. doi:10.1109/FMCAD.2014.6987607
- [24] Akash Lal, Shaz Qadeer, and Shuvendu K. Lahiri. 2012. A Solver for Reachability Modulo Theories. In *Computer Aided Verification - 24th International Conference, CAV 2012, Berkeley, CA, USA, July 7-13, 2012 Proceedings (Lecture Notes in Computer Science, Vol. 7358)*, P. Madhusudan and Sanjit A. Seshia (Eds.). Springer, 427–443. doi:10.1007/978-3-642-31424-7_32
- [25] Chris Lattner and Vikram S. Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *2nd IEEE / ACM International Symposium on Code Generation and Optimization (CGO 2004), 20-24 March 2004, San Jose, CA, USA*. IEEE Computer Society, 75–88. doi:10.1109/CGO.2004.1281665
- [26] Lian Li, Cristina Cifuentes, and Nathan Keynes. 2011. Boosting the Performance of Flow-Sensitive Points-to Analysis Using Value Flow. In *SIGSOFT/FSE’11 19th ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE-19) and ESEC’11: 13th European Software Engineering Conference (ESEC-13), Szeged, Hungary, September 5-9, 2011*, Tibor Gyimóthy and Andreas Zeller (Eds.). ACM, 343–353. doi:10.1145/2025113.2025160
- [27] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.
- [28] Benjamin Livshits, Manu Sridharan, Yannis Smaragdakis, Ondrej Lhoták, José Nelson Amaral, Bor-Yuh Evan Chang, Samuel Z. Guyer, Uday P. Khedker, Anders Møller, and Dimitrios Vardoulakis. 2015. In Defense of Soundness: A Manifesto. *Commun. ACM* 58, 2 (2015), 44–46. doi:10.1145/2644805
- [29] Ravi Mangal, Xin Zhang, Aditya V. Nori, and Mayur Naik. 2015. A User-Guided Approach to Program Analysis. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2015, Bergamo, Italy, August 30 - September 4, 2015*, Elisabetta Di Nitto, Mark Harman, and Patrick Heymans (Eds.). ACM, 462–473. doi:10.1145/2786805.2786851
- [30] Scott McPeak, Charles-Henri Gros, and Murali Krishna Ramanathan. 2013. Scalable and Incremental Software Bug Detection. In *Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering, ESEC/FSE’13, Saint Petersburg, Russian Federation, August 18-26, 2013*, Bertrand Meyer, Luciano Baresi, and Mira Mezini (Eds.). ACM, 554–564. doi:10.1145/2491411.2501854
- [31] Sixiang Peng, Chenyang Sun, Wei Chen, Bowen Zhang, and Charles Zhang. 2026. SPONGE: Adaptive Boundary-Anchored Indexing for Online Value-Flow Queries Artifacts. Zenodo. doi:10.5281/zenodo.21777386
- [32] G. D. Plotkin. 1975. Call-by-Name, Call-by-Value and the λ -Calculus. *Theoretical Computer Science* 1, 2 (Dec. 1975), 125–159. doi:10.1016/0304-3975(75)90017-1
- [33] Cap’n Proto. 2026. Cap’n Proto. <https://capnproto.org/>.
- [34] Tamjid Al Rahat, Yu Feng, and Yuan Tian. 2022. Cerberus: Query-driven Scalable Vulnerability Detection in OAuth Service Provider Implementations. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM, 2459–2473. doi:10.1145/3548606.3559381
- [35] Thomas W. Reps. 1998. Program Analysis via Graph Reachability. *Inf. Softw. Technol.* 40, 11-12 (1998), 701–726. doi:10.1016/S0950-5849(98)00093-7
- [36] Thomas W. Reps, Susan Horwitz, and Shmuel Sagiv. 1995. Precise Interprocedural Dataflow Analysis via Graph Reachability. In *Conference Record of POPL’95: 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming*

- Languages*, San Francisco, California, USA, January 23-25, 1995, Ron K. Cytron and Peter Lee (Eds.). ACM Press, 49–61. doi:10.1145/199448.199462
- [37] Thomas W. Reps, Stefan Schwoon, and Somesh Jha. 2003. Weighted Pushdown Systems and Their Application to Interprocedural Dataflow Analysis. In *Static Analysis, 10th International Symposium, SAS 2003, San Diego, CA, USA, June 11-13, 2003, Proceedings (Lecture Notes in Computer Science, Vol. 2694)*, Radhia Cousot (Ed.). Springer, 189–213. doi:10.1007/3-540-44898-5_11
- [38] Caitlin Sadowski, Jeffrey van Gogh, Ciera Jaspan, Emma Söderberg, and Collin Winter. 2015. Tricorder: Building a Program Analysis Ecosystem. In *37th IEEE/ACM International Conference on Software Engineering, ICSE 2015, Florence, Italy, May 16-24, 2015, Volume 1*, Antonia Bertolino, Gerardo Canfora, and Sebastian G. Elbaum (Eds.). IEEE Computer Society, 598–608. doi:10.1109/ICSE.2015.76
- [39] Shmuel Sagiv, Thomas W. Reps, and Susan Horwitz. 1996. Precise Interprocedural Dataflow Analysis with Applications to Constant Propagation. *Theor. Comput. Sci.* 167, 1&2 (1996), 131–170. doi:10.1016/0304-3975(96)00072-2
- [40] Edward J. Schwartz, Thanassis Avgerinos, and David Brumley. 2010. All You Ever Wanted to Know about Dynamic Taint Analysis and Forward Symbolic Execution (but Might Have Been Afraid to Ask). In *31st IEEE Symposium on Security and Privacy, SP 2010, 16-19 May 2010, Berkeley/Oakland, California, USA*. IEEE Computer Society, 317–331. doi:10.1109/SP.2010.26
- [41] Qingkai Shi, Yongchao Wang, Peisen Yao, and Charles Zhang. 2022. Indexing the Extended Dyck-CFL Reachability for Context-Sensitive Program Analysis. *Proc. ACM Program. Lang.* 6, OOPSLA2 (2022), 1438–1468. doi:10.1145/3563339
- [42] Qingkai Shi, Rongxin Wu, Gang Fan, and Charles Zhang. 2020. Conquering the Extensional Scalability Problem for Value-Flow Analysis Frameworks. In *ICSE '20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June - 19 July, 2020*, Gregg Rothermel and Doo-Hwan Bae (Eds.). ACM, 812–823. doi:10.1145/3377811.3380346
- [43] Qingkai Shi, Xiao Xiao, Rongxin Wu, Jinguo Zhou, Gang Fan, and Charles Zhang. 2018. Pinpoint: Fast and Precise Sparse Value Flow Analysis for Million Lines of Code. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018, Philadelphia, PA, USA, June 18-22, 2018*, Jeffrey S. Foster and Dan Grossman (Eds.). ACM, 693–706. doi:10.1145/3192366.3192418
- [44] Qingkai Shi, Peisen Yao, Rongxin Wu, and Charles Zhang. 2021. Path-Sensitive Sparse Analysis without Path Conditions. In *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 2021*, Stephen N. Freund and Eran Yahav (Eds.). ACM, 930–943. doi:10.1145/3453483.3454086
- [45] Qingkai Shi and Charles Zhang. 2020. Pipelining Bottom-up Data Flow Analysis. In *ICSE '20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June - 19 July, 2020*, Gregg Rothermel and Doo-Hwan Bae (Eds.). ACM, 835–847. doi:10.1145/3377811.3380425
- [46] Johannes Späth, Karim Ali, and Eric Bodden. 2019. Context-, Flow-, and Field-Sensitive Data-Flow Analysis Using Synchronized Pushdown Systems. *Proc. ACM Program. Lang.* 3, POPL (2019), 48:1–48:29. doi:10.1145/3290361
- [47] Johannes Späth, Lisa Nguyen Quang Do, Karim Ali, and Eric Bodden. 2016. Boomerang: Demand-Driven Flow- and Context-Sensitive Pointer Analysis for Java. In *30th European Conference on Object-Oriented Programming, ECOOP 2016, Rome, Italy, July 18-22, 2016 (LIPICs, Vol. 56)*, Shriram Krishnamurthi and Benjamin S. Lerner (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 22:1–22:26. doi:10.4230/LIPICs.ECOOP.2016.22
- [48] Manu Sridharan, Denis Gopan, Lexin Shan, and Rastislav Bodík. 2005. Demand-Driven Points-to Analysis for Java. In *Proceedings of the 20th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2005, October 16-20, 2005, San Diego, CA, USA*, Ralph E. Johnson and Richard P. Gabriel (Eds.). ACM, 59–76. doi:10.1145/1094811.1094817
- [49] Yulei Sui and Jingling Xue. 2016. On-Demand Strong Update Analysis via Value-Flow Refinement. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2016, Seattle, WA, USA, November 13-18, 2016*, Thomas Zimmermann, Jane Cleland-Huang, and Zhendong Su (Eds.). ACM, 460–473. doi:10.1145/2950290.2950296
- [50] Yulei Sui and Jingling Xue. 2016. SVF: Interprocedural Static Value-Flow Analysis in LLVM. In *Proceedings of the 25th International Conference on Compiler Construction, CC 2016, Barcelona, Spain, March 12-18, 2016*, Ayaz Zaks and Manuel V. Hermenegildo (Eds.). ACM, 265–266. doi:10.1145/2892208.2892235
- [51] Yulei Sui, Ding Ye, and Jingling Xue. 2012. Static Memory Leak Detection Using Full-Sparse Value-Flow Analysis. In *International Symposium on Software Testing and Analysis, ISSTA 2012, Minneapolis, MN, USA, July 15-20, 2012*, Mats Per Erik Heimdahl and Zhendong Su (Eds.). ACM, 254–264. doi:10.1145/2338965.2336784
- [52] Wensheng Tang, Dejun Dong, Shijie Li, Chengpeng Wang, Peisen Yao, Jinguo Zhou, and Charles Chuan Zhang. 2024. Octopus: Scaling Value-Flow Analysis via Parallel Collection of Realizable Path Conditions. *ACM Trans. Softw. Eng. Methodol.* 33, 3 (2024), 66:1–66:33. doi:10.1145/3632743
- [53] Julien Vanegue and Shuvendu K. Lahiri. 2013. Towards Practical Reactive Security Audit Using Extended Static Checkers. In *2013 IEEE Symposium on Security and Privacy, SP 2013, Berkeley, CA, USA, May 19-22, 2013*. IEEE Computer Society, 33–47. doi:10.1109/SP.2013.12

- [54] Claire Wang, Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. QLCoder: A Query Synthesizer For Static Analysis of Security Vulnerabilities. *CoRR* abs/2511.08462 (2025). arXiv:2511.08462 doi:10.48550/ARXIV.2511.08462
- [55] Chao Wang, Li Lin, Chengpeng Wang, Jiafeng Huang, CongXia Wu, and Rongxin Wu. 2025. ReachCheck: Compositional Library-Aware Call Graph Reachability Analysis in the IDEs. *ACM Trans. Softw. Eng. Methodol.* (Sept. 2025). doi:10.1145/3767166
- [56] Kai Wang, Aftab Hussain, Zhiqiang Zuo, Guoqing Xu, and Ardalan Amiri Sani. 2017. Graspan: A Single-machine Disk-based Graph System for Interprocedural Static Analyses of Large-scale Systems Code. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2017, Xi'an, China, April 8-12, 2017*, Yunji Chen, Olivier Temam, and John Carter (Eds.). ACM, 389–404. doi:10.1145/3037697.3037744
- [57] Rongxin Wu, Yuxuan He, Jiafeng Huang, Chengpeng Wang, Wensheng Tang, Qingkai Shi, Xiao Xiao, and Charles Zhang. 2024. LibAlchemy: A Two-Layer Persistent Summary Design for Taming Third-Party Libraries in Static Bug-Finding Systems. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 105:1–105:13. doi:10.1145/3597503.3639132
- [58] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and Discovering Vulnerabilities with Code Property Graphs. In *2014 IEEE Symposium on Security and Privacy, SP 2014, Berkeley, CA, USA, May 18-21, 2014*. IEEE Computer Society, 590–604. doi:10.1109/SP.2014.44
- [59] Guowei Yang, Corina S. Pasareanu, and Sarfraz Khurshid. 2012. Memoized Symbolic Execution. In *International Symposium on Software Testing and Analysis, ISSTA 2012, Minneapolis, MN, USA, July 15-20, 2012*, Mats Per Erik Heimdahl and Zhendong Su (Eds.). ACM, 144–154. doi:10.1145/2338965.2336771
- [60] Danfeng Zhang and Andrew C. Myers. 2014. Toward General Diagnosis of Static Errors. In *The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '14, San Diego, CA, USA, January 20-21, 2014*, Suresh Jagannathan and Peter Sewell (Eds.). ACM, 569–582. doi:10.1145/2535838.2535870
- [61] Qirun Zhang, Michael R. Lyu, Hao Yuan, and Zhendong Su. 2013. Fast Algorithms for Dyck-CFL-reachability with Applications to Alias Analysis. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13, Seattle, WA, USA, June 16-19, 2013*, Hans-Juergen Boehm and Cormac Flanagan (Eds.). ACM, 435–446. doi:10.1145/2491956.2462159

Received 2026-03-17; accepted 2026-06-10